

Chapter 3 Arithmetic for Computers

3.1 Introduction

- Computer words are composed of bits; thus words can be represented as binary numbers.

- What about fractions and other real numbers?
- What happen if an operations creates a number bigger than can be represented
- And underlying these questions is a mystery: How does hardware really multiply or divide numbers?

1、计算机如何实现运算？（加、减、乘、除）

2、计算机如何表示数？（整数，正、负数，小数等）

3、计算机如何处理溢出、舍去、舍入等？

4、计算机硬件如何实现运算？

3.2 Addition and subtraction

- 先看加、减运算。
- 二进制运算过程。

Addition & subtraction

- Adding bit by bit, carries -> next digit

0000 0111	7_{10}
+0000 0110	6_{10}
0000 1101	13_{10}

- Subtraction
  - Directly
  - Addition of 2's complement

0000 0111	7_{10}
- 0000 0110	6_{10}
0000 0001	1_{10}

0000 0111	7_{10}
+1111 1010	-6_{10}
0000 0001	1_{10}

Overflow

- The sum of two numbers can exceed any representation

$$\begin{array}{rr} 1111\ 1111 & 255_{10} \\ +1111\ 1010 & 250_{10} \\ \hline 1\ 1111\ 1001 & 249_{10} \end{array}$$

- The difference of two numbers can exceed any representation
- 2's complement:

  Numbers change

  sign and size

$$\begin{array}{rr} 1000\ 0001 & -127_{10} \\ +1111\ 1110 & -2_{10} \\ \hline 0111\ 1111 & +127_{10} \end{array}$$

Overflow

- General overflow conditions

Operation	Conditions		Result
A + B	A > 0	B > 0	< 0
A + B	A < 0	B < 0	> 0
A - B	A > 0	B < 0	< 0
A - B	A < 0	B > 0	>0

Conditional branch that test for Overflow

- BVS (程序指针相对寻址)
  - branch if overflow set
- BVC （程序指针相对寻址)
  - branch if overflow clear

Overflow

- Reaction on overflow（溢出的反应）
  - Ignore ?
  - Reaction of the OS
  - Signalling to application (Ada, Fortran,...)
- Hardware detection in the ALU
- Generation of an exception (interrupt)
- Save the instruction address (not PC) in special register EPC
- Jump to specific routine in OS
  - Correct & return to program
  - Return to program with error code
  - Abort program（中断程序）

Overflow

- Overflows in signed arithmetic instructions cause exceptions:
  - add
  - add immediate
  - subtract
- Overflows in unsigned arithmetic instructions don't cause exceptions:
  - add unsigned
  - add immediate unsigned
  - Subtract unsigned
  - Handle with care!

summary

- Allow two's complement integer arithmetic on variable declared Byte and half
  - Load with LB/LH.（一个字节与半个字节）

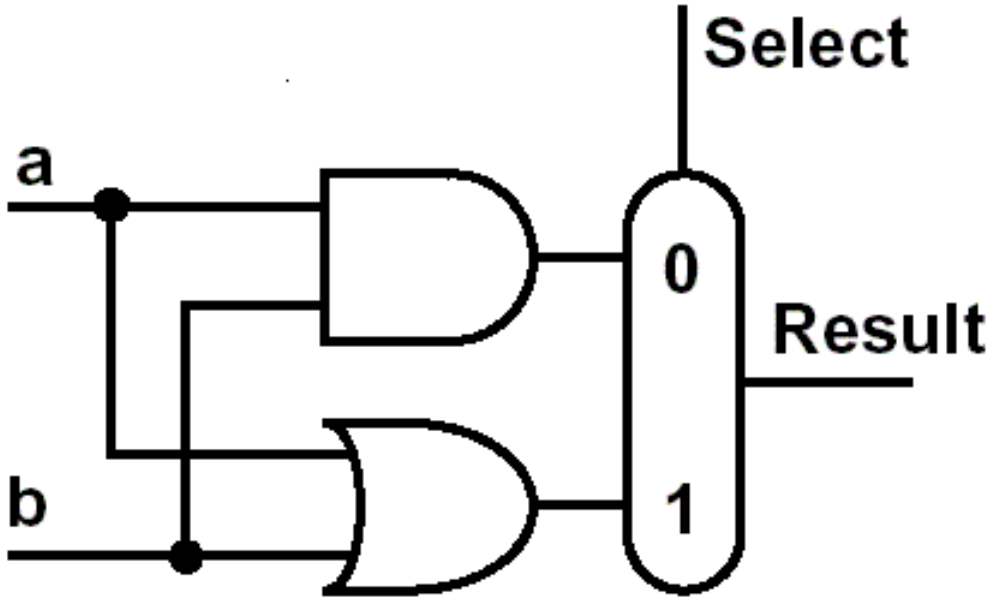
补充-附录A.5, A.6

- 1、 constructing a basic arithmetic logic unit
- 2、 faster addition: carry lookahead

- 电路如何实现加、减运算？
- ALU设计

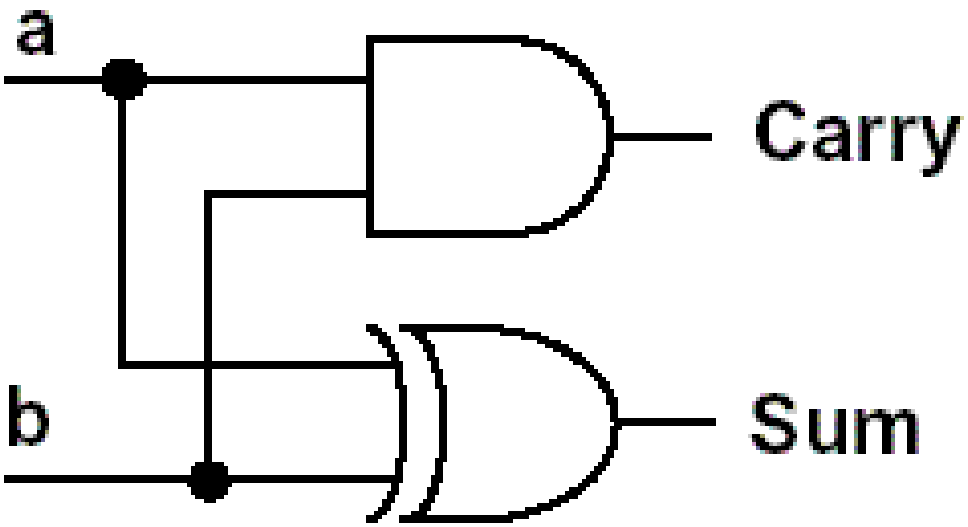
Constructing an ALU（算术逻辑运算单元）

- Step by step: build a single bit ALU and expand it to the desired width
- First function: logic AND and OR



A half adder

- Sum = $\bar{a}\, b + a\, \bar{b}$
- Carry = a b



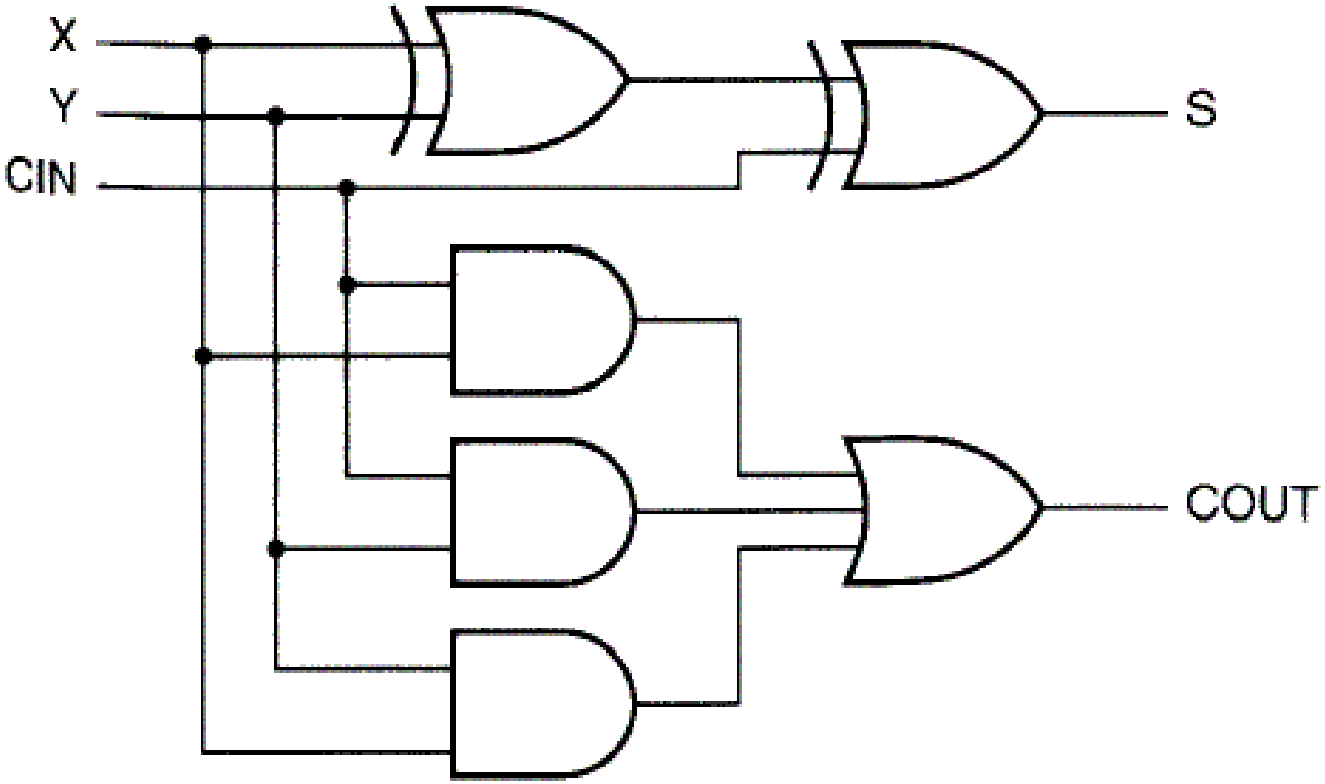
A full adder

- Accepts a carry in
- Sum = A xor B xor CarryIn
- CarryOut = B CarryIn + A CarryIn + A B

Inputs			Outputs		Comments
A	B	CarryIn	CarryOut	Sum	(two)
0	0	0	0	0	0+0+0=00
0	0	1	0	1	0+0+1=01
0	1	0	0	1	0+1+0=01
0	1	1	1	0	0+1+1=10
1	0	0	0	1	1+0+0=01
1	0	1	1	0	1+0+1=10
1	1	0	1	0	1+1+0=10
1	1	1	1	1	1+1+1=11

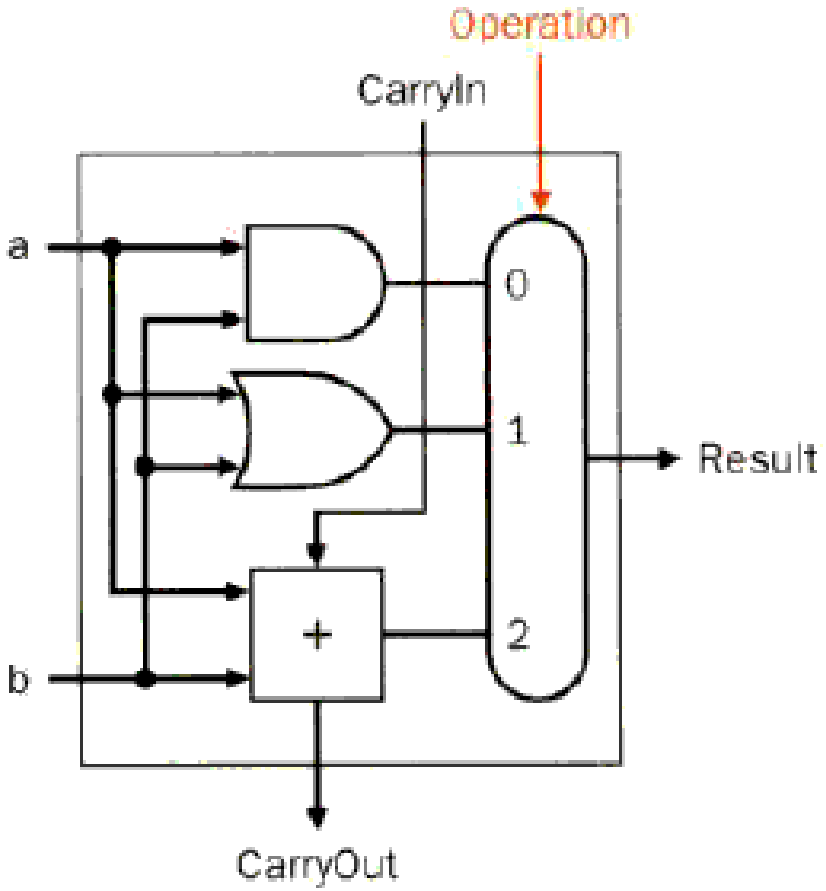
Full adder

- Full adder in 2-level design



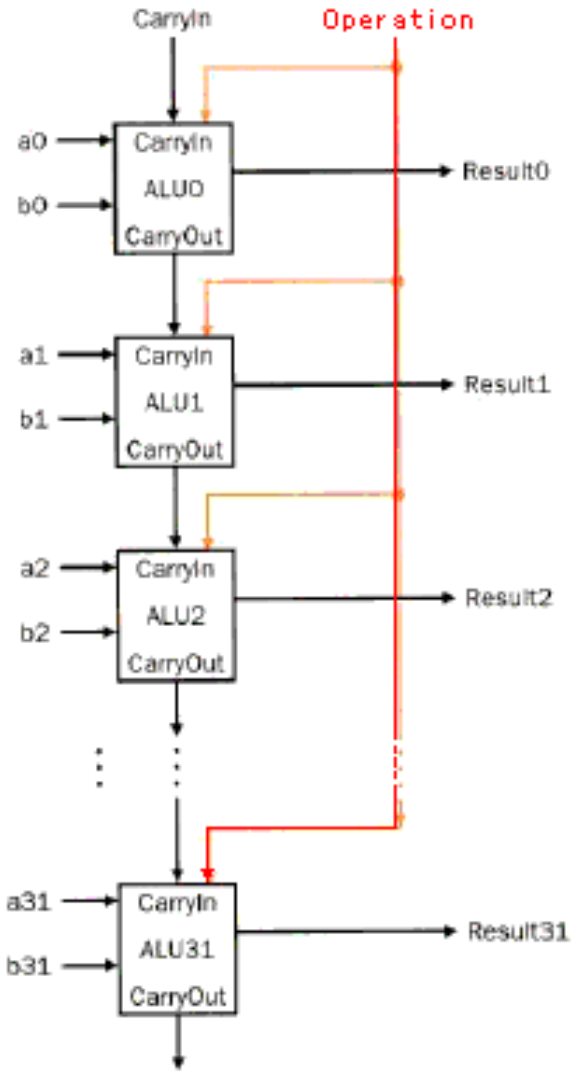
1 bit ALU

- ALU
  - AND
  - OR
  - ADD
- Cascade Element



Basic 32 bit ALU

- Inputs parallel
- Carry is cascaded
- Ripple carry adder
- Slow, but simple
- 1st Carry In = 0



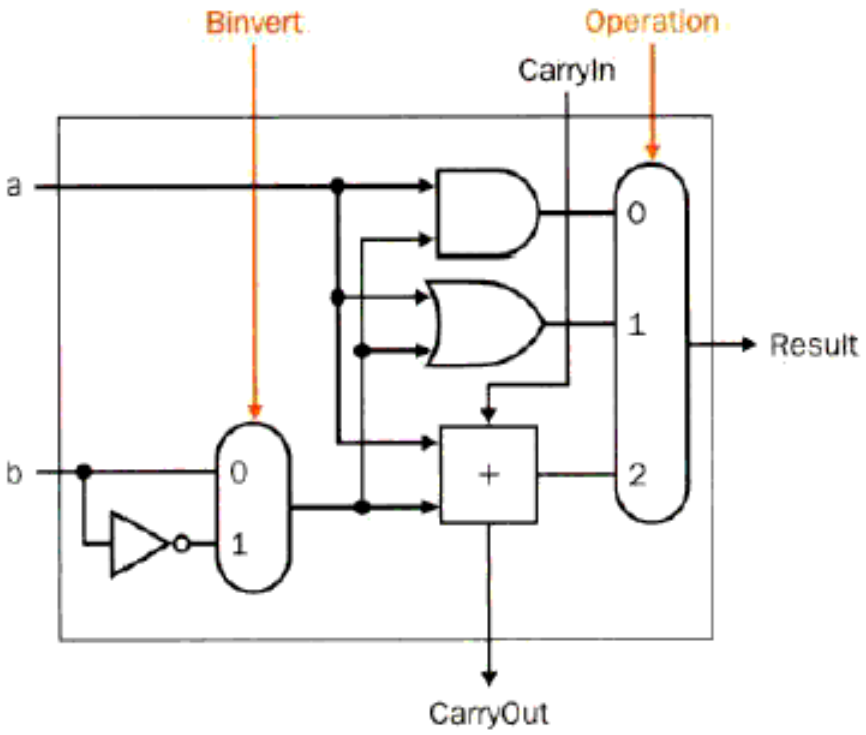
Extended 1 bit ALU

- Subtraction

  a - b

  - Inverting b
  - 1st CarryIn= 1

采用补码方法

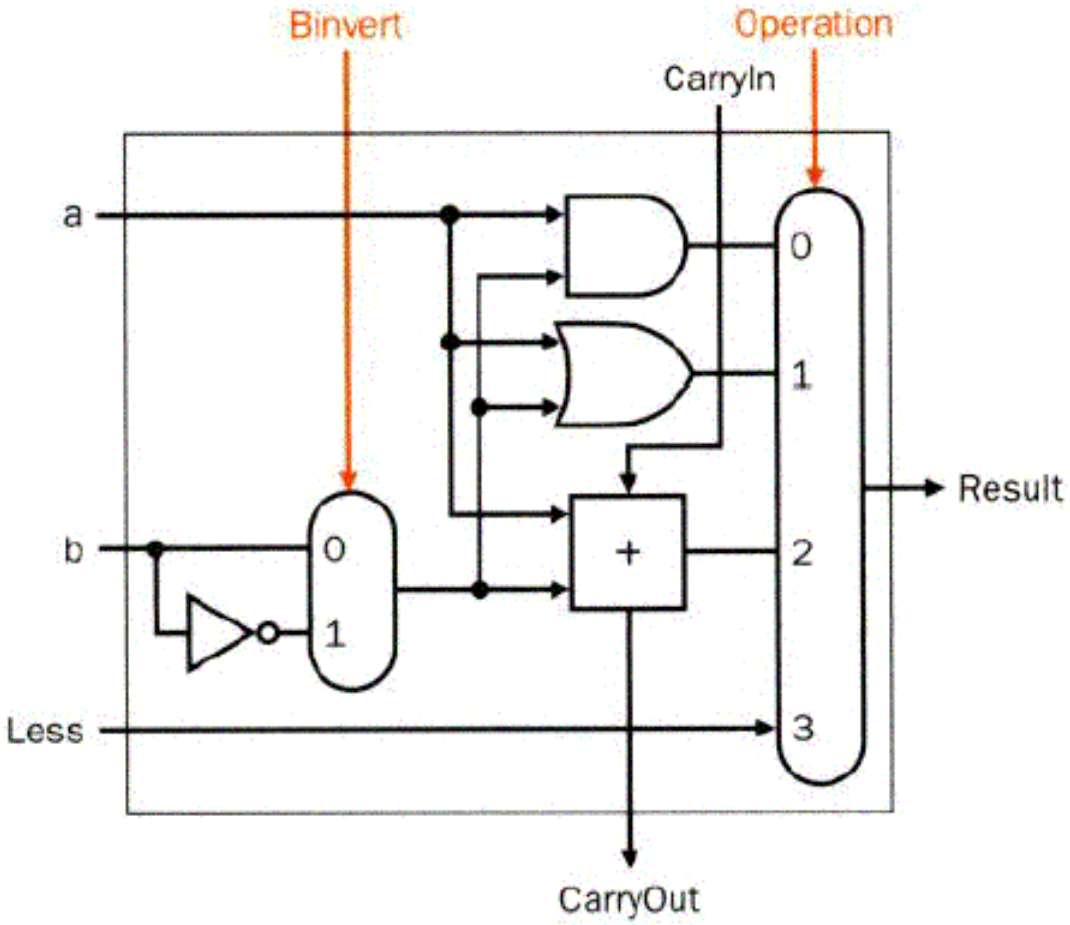


Extended 1 bit ALU

- Functions
  - AND
  - OR
  - Add
  - Subtract
- Missing: comparison
  - Slt rd,rs,rt
  - If rs < rt, rd=1, else rd=0
  - All bits = 0 except the least significant(比较结果只要一位）
  - Subtraction (rs - rt), if the result is negative -> rs < rt
  - Use of sign bit as indicator

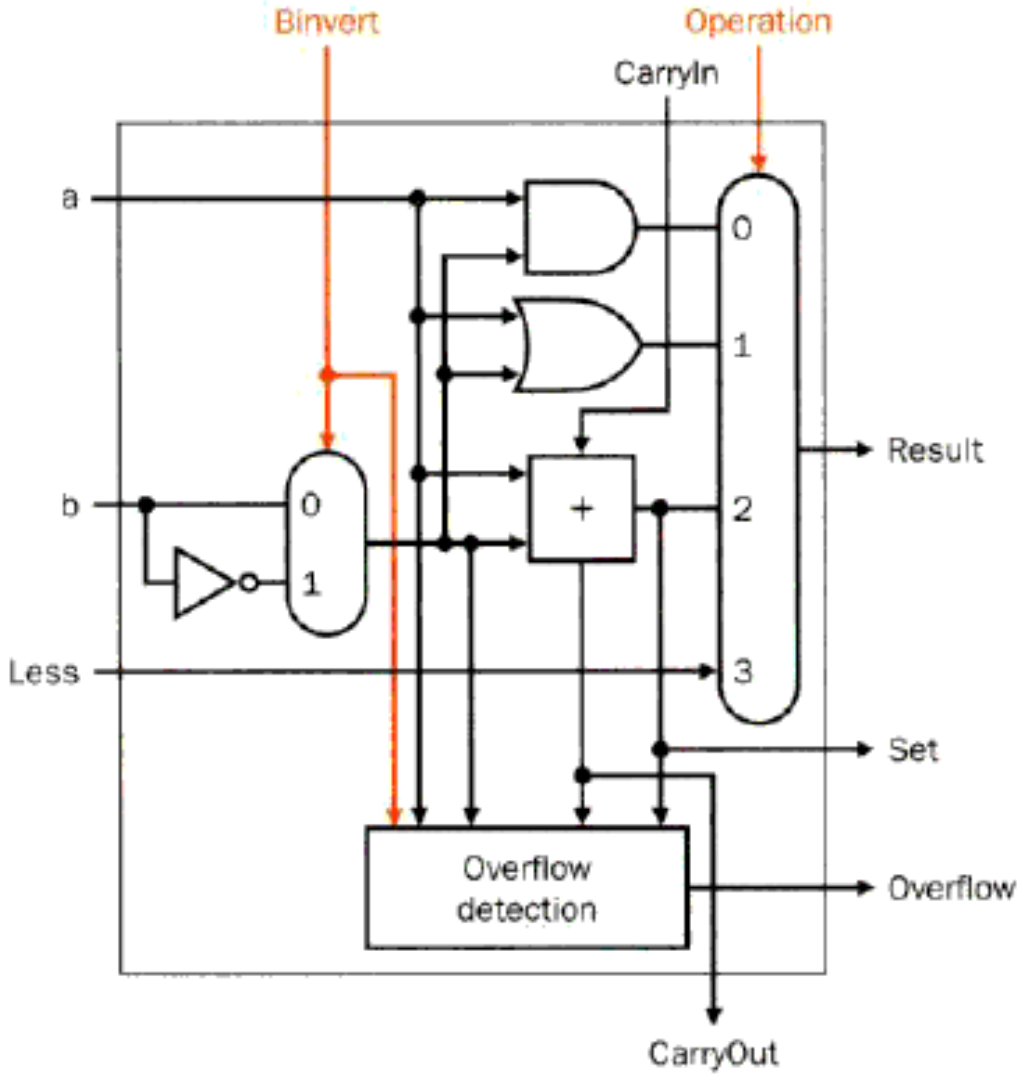
Extended 1 bit ALU

- ALU bit with input for Less data



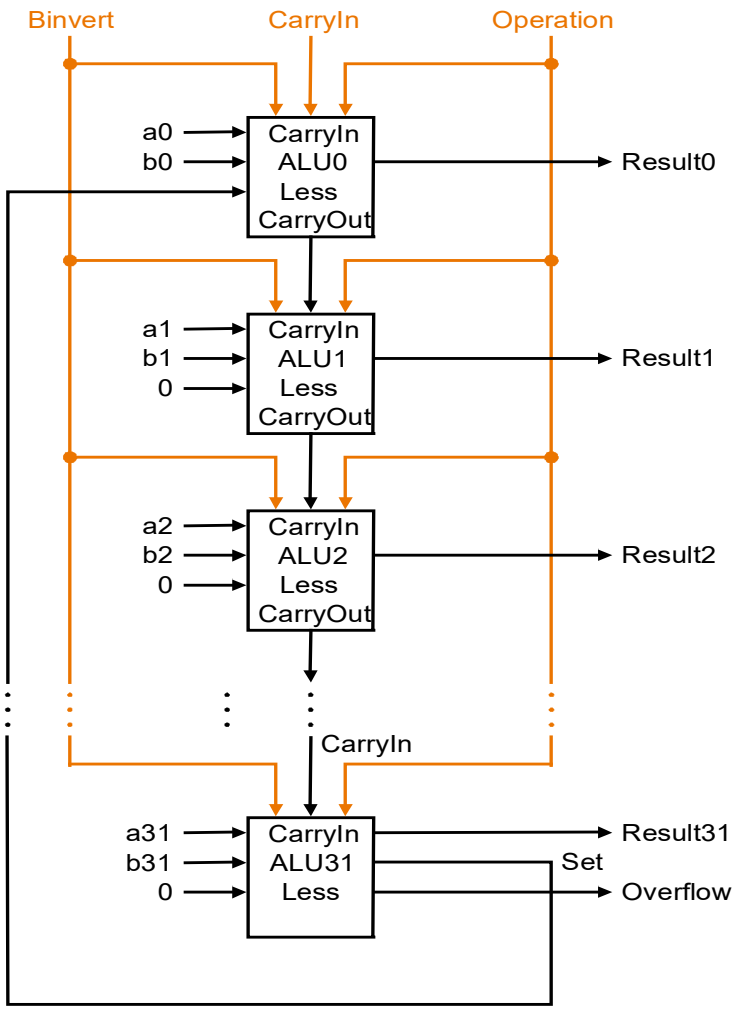
Most significant bit（最高位ALU）

- Set for comparison
- Overflow detect



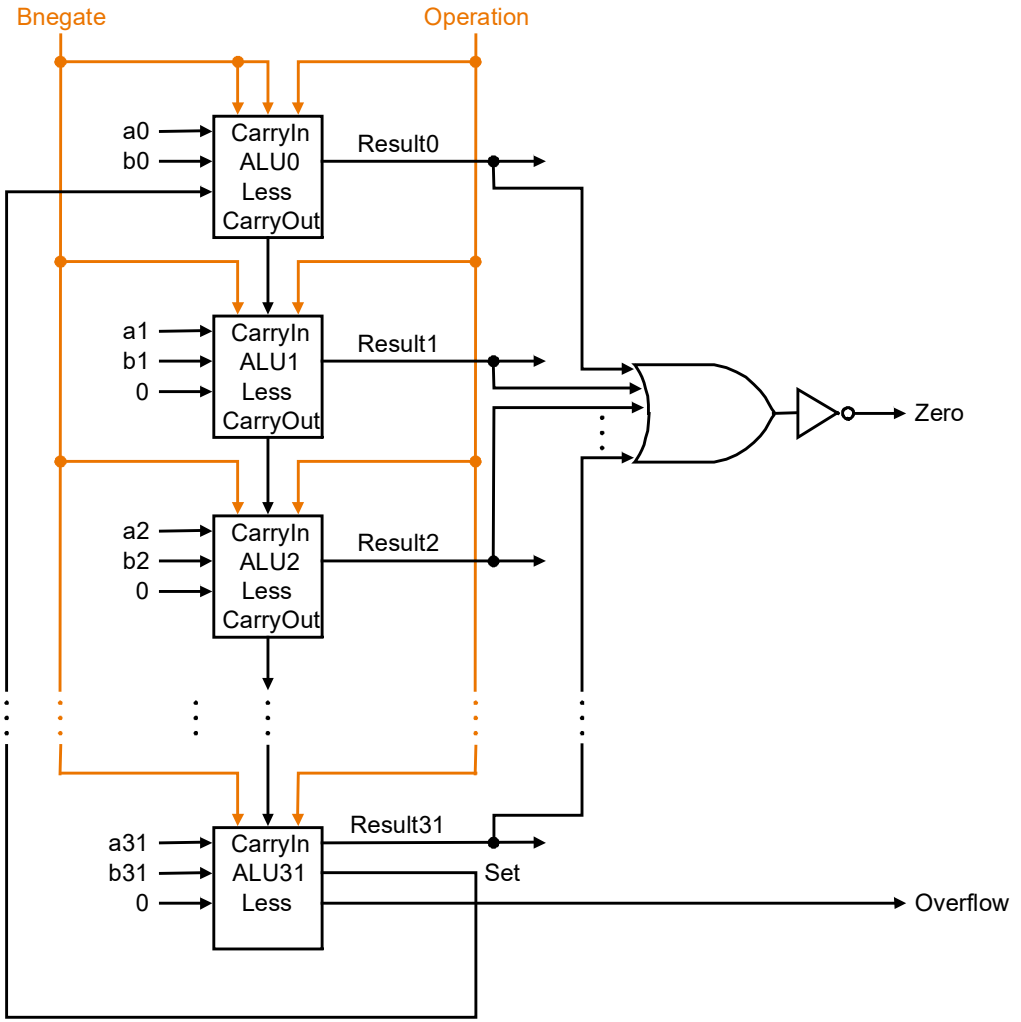
Complete ALU

- Input
  - A
  - B
- Control lines
  - Binvert
  - Operation
  - Carryin
- Output
  - Result
  - Overflow



Complete ALU

- Add a Zero detector

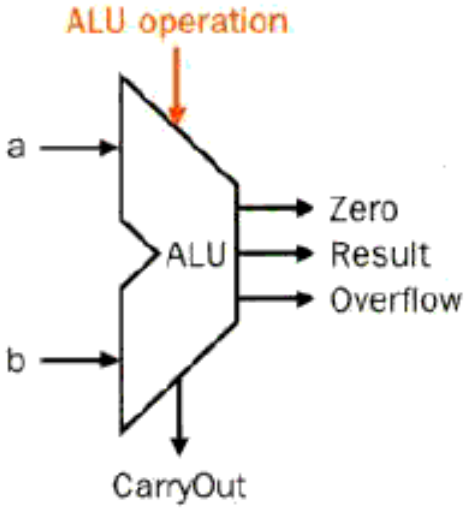


ALU symbol & control

- Function table

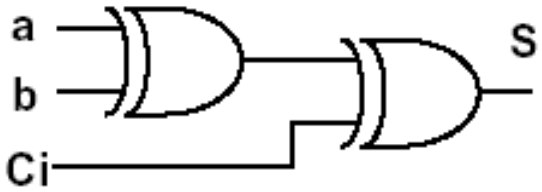
ALU Control Lines	Function
000	And
001	Or
010	Add
110	Sub
111	Set on less than

- Symbol of the ALU

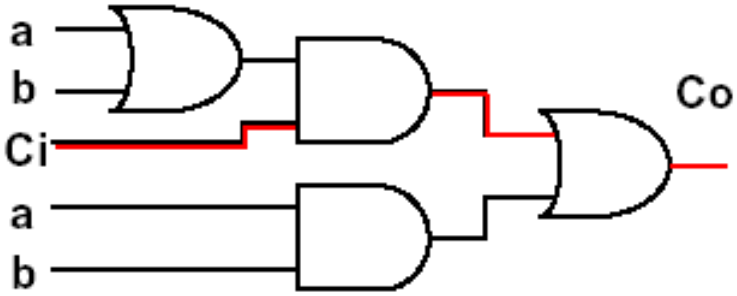


Speed considerations

- Previously used: ripple carry adder
- Delay for the sum: two units

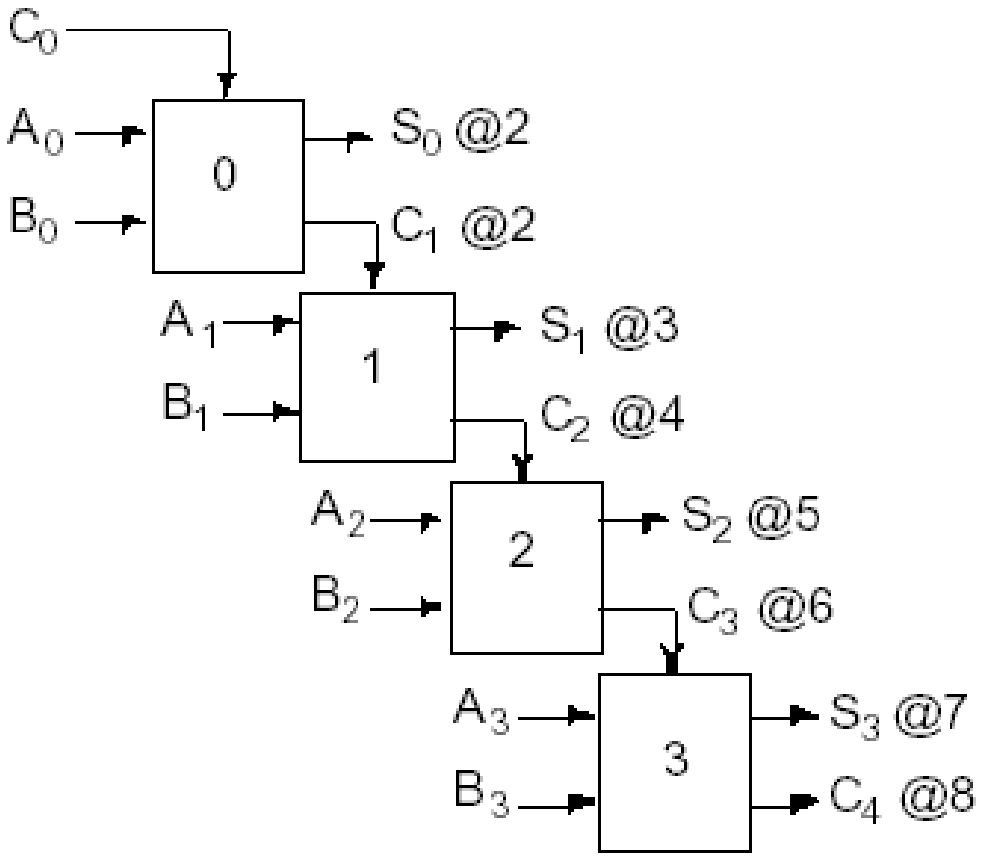


- Delay for the carry: two - three units



Speed considerations

- Delay of one adder
  - 2 time units
- Total delay for stages:
  2n unit delays
- Not appropriate for high speed application



Fast adders

- All functions can be represented in 2-level logic.
- But:
  - The number of inputs of the gates would drastically rise
- Target:

  Optimum between speed and size

Carry look ahead adder (CLA)

- Separation of
  - add operation
  - carry calculation
- Factorization
  - Ci+1 = (bi * ci) + (ai * ci) + (ai * bi)
    =(ai * bi) + (ai + bi) * ci
  - Generate gi = ai * bi
  - Propagate pi = ai + bi

Carry look ahead adder

- Ci+1 = gi + pi * ci
- Carry generate: gi = ai * bi
  - If a and b are '1' ->
    we always have a carryout independent of ci
- Carry propagate: pi = ai+ bi
  - If only one of a and b is '1' ->
    the carry out depends on the carry in
  - pi propagates the carry

Four bit carry look ahead adder

- c1 = g0 + (p0 * c0)
- c2 = g1 + p1*c1 = g1 + (p1 * g0) + (p1 * p0 * c0)
- c3 = g2 + p2*c2 = g2 + (p2 * g1) + (p2 * p1 * g0) + (p2 * p1 * p0 * c0)
- c4 = g3 + p3*c3 = g3 + (p3 * g2) + (p3 * p2 * g1) +(p3 * p2 * p1 * g0) + (p3 * p2 * p1 * p0 * c0)

COMMENT:

This kind of adder will be faster than the ripple carry adder, and smaller than the adder with the two-level logic.

PROBLEM:

If the number of the adder bits is very large, this kind of adder will be too large. So we must seek more efficient ways.

折中解决方案：分级，级内四位一组，组内并行，但级间串行。

Four bit carry look ahead adder

Let's consider a 16-bit adder.

Divide 16 bits into 4 groups. Each group has 4 bits.

As we know:

c4 = g3 + p3*g2 + p3*p2*g1 + p3*p2*p1*g0 + p3*p2*p1*p0*c0

So,we can get the following:

c8 = g7 + p7*g6 + p7*p6*g5 + p7*p6*p5*g4 + p7*p6*5*p4*c4

c12 =
g11+p11*g10+p11*p10*g9+p11*p10*p9*g8+p11*p10*p9*p8*c8

c16=g15+p15*g14+p15*p14*g13+p15*p14*p13*g12+p15*p14*p13*p12*c12

Assume:

G0= g3 + p3*g2 + p3*p2 *g1 +p3*p2*p1*g0

G1= g7 + p7*g6 + p7*p6*g5 + p7*p6*p5*g4

G2= g11+p11*g10+p11*p10*g9+p11*p10*p9*g8

G3= g15+p15*g14+p15*p14*g13+p15*p14*p13*g12

P0= p3 * p2 * p1 * p0

P1= p7 * p6 * p5 * p4

P2= p11 * p10 * p9 * p8

P3= p15 * p14 * p13 * p12

Four bit carry look ahead adder

Then we get:

c4=G0+P0*c0 ; c8=G1+P1*c4

c12=G2+P2*c8 ; c16=G3+P3*c12

Assume:C1=c4,C2=c8,C3=c12,C4=c16

Then:

C1=G0+P0*c0 ; C2=G1+P1*C1

C3=G2+P2*C2 ; C4=G3+P3*C3

And,we can further get:

C1=G0+P0*c0 ;

C2= G1+P1*C1 = G1+P1*G0+ P1* P0*c0

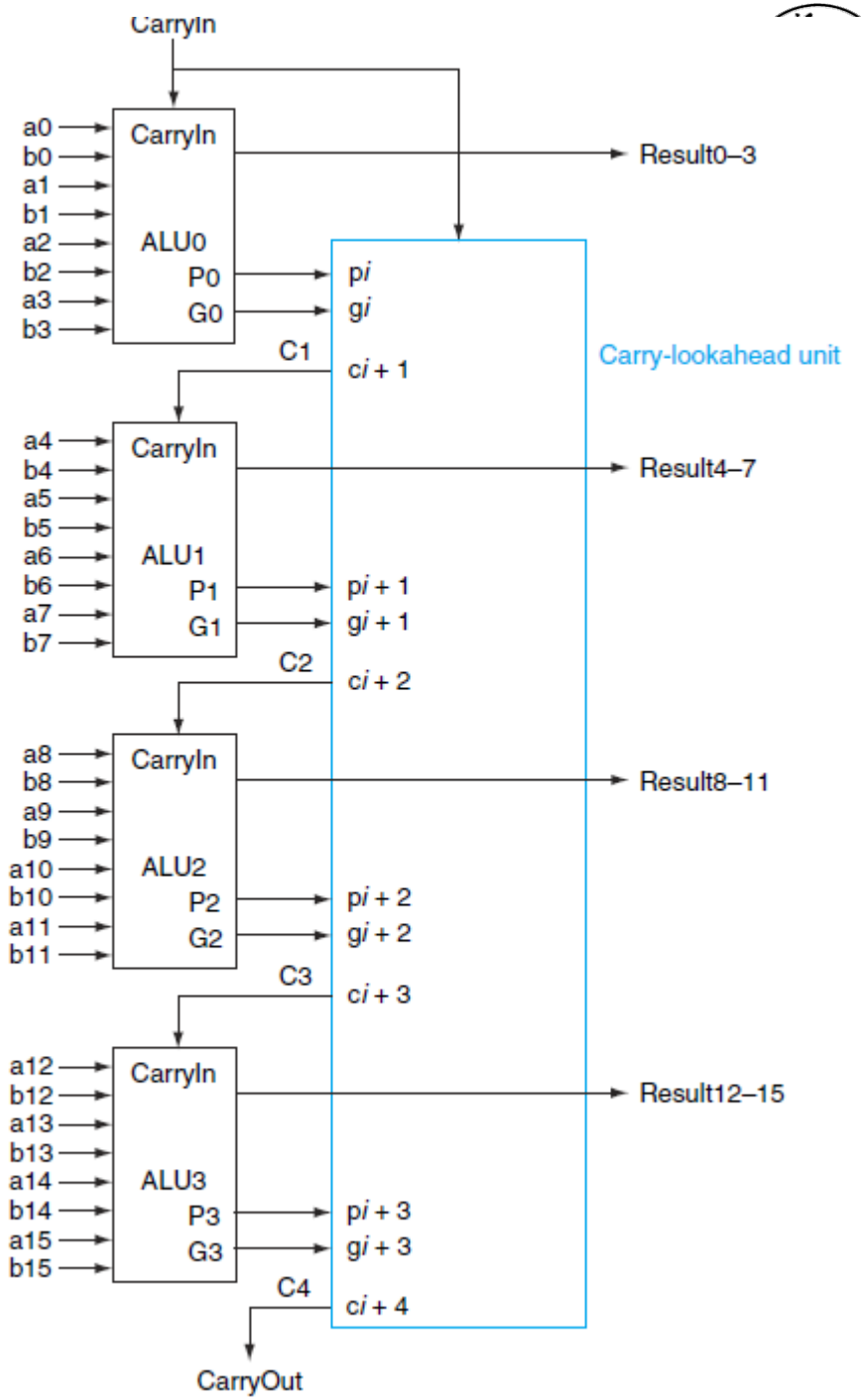
C3=G2+P2*C2 = G2+P2*G1 + P2* P1*G0+ P2* P1* P0*c0

C4=G3+P3*C3=

G3+P3*G2+P3*P2*G1+P3*P2*P1*G0+P3*P2*P1* P0*c0

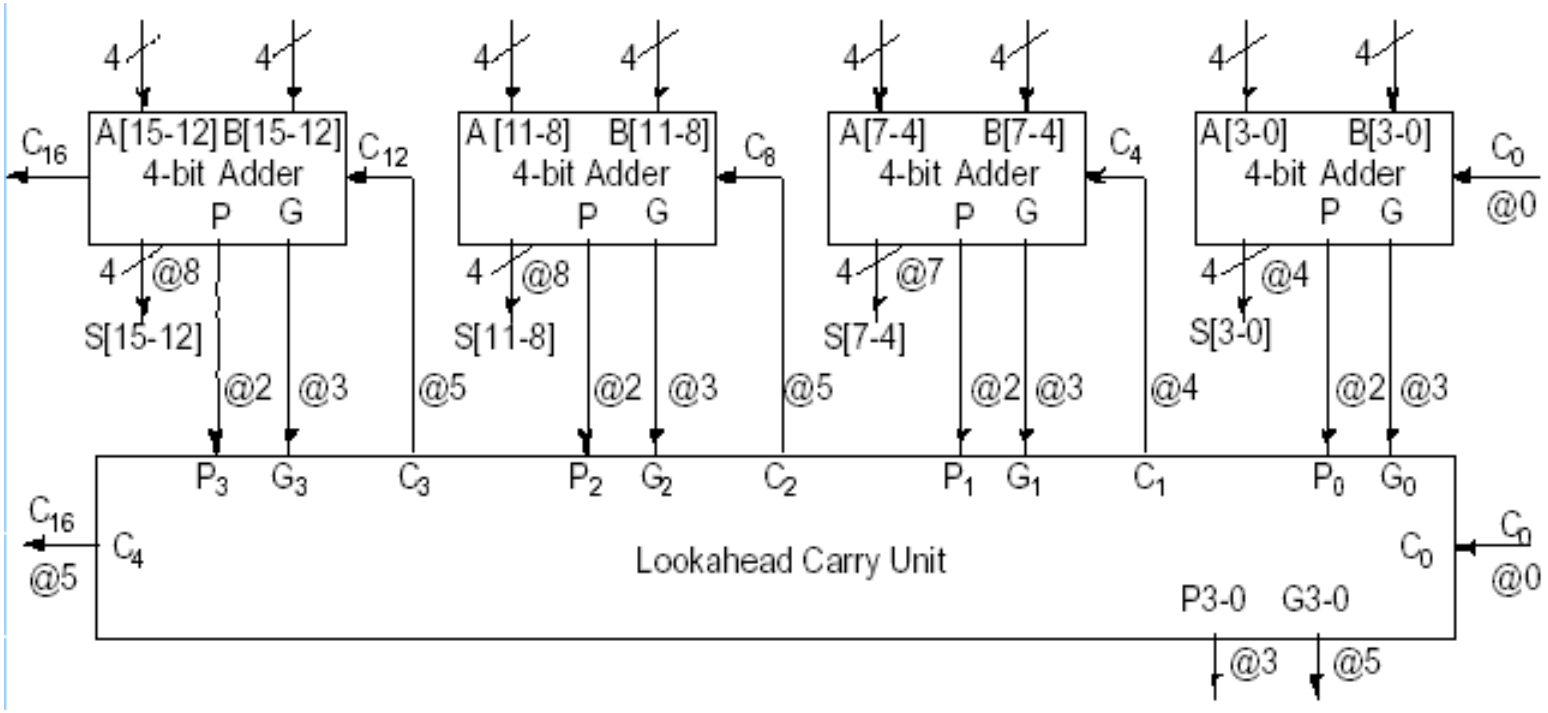
二级并行进位16位加法器（用4个4位并行进位加法器，4位并行进位加法器间再用并行进位）

Four 4-bit ALUs using carry lookahead to form a 16-bit adder.



Hybrid CLA + Ripple carry

- Realisation:
  - Ripple carry adders and
  - Carry look ahead logic



3.3 Multiplication

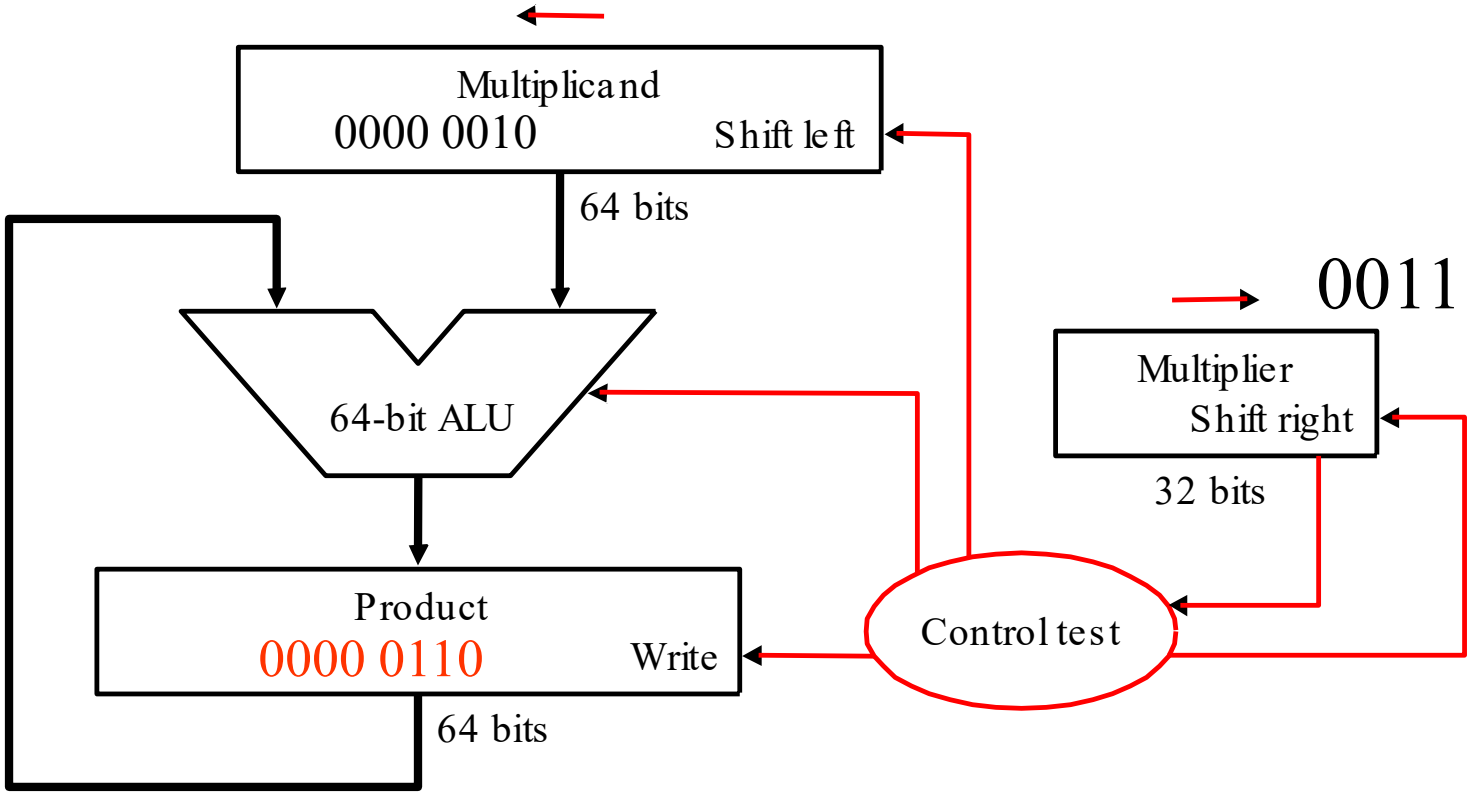
- Binary multiplication
  Multiplicand * Multiplier

```
        1000 * 1001
            1000
           1001
          ------
            1000
           0000
          0000
         1000
Product  1001000
```

- Look at current bit position
  - If multiplier is 1
    - then add multiplicand
    - Else add 0
  - shift multiplicand left by 1 bit

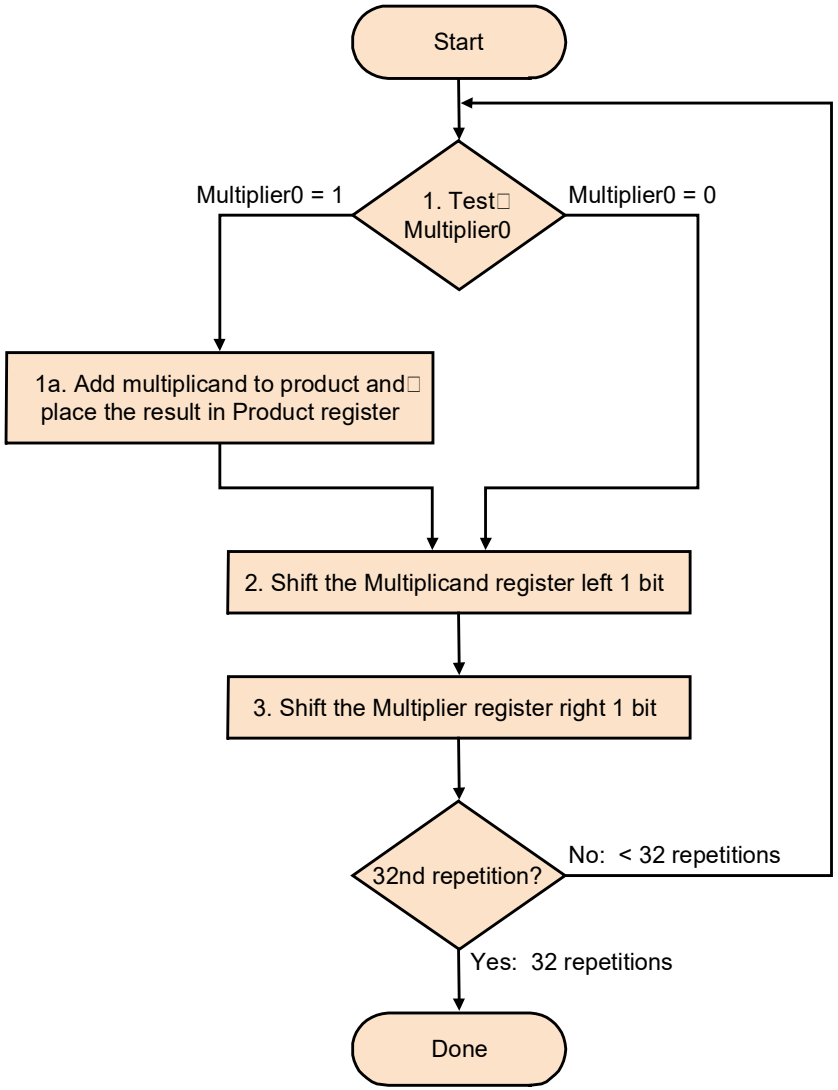
Multiplier Sequential Version

- 32 bits: multiplier
- 64 bits: multiplicand, product, ALU
- 0010*0011



Multiplier Sequential Version

- Requires 32 iterations
  - Addition
  - Shift
  - Comparison
- Almost 100 cycles
- Very big, Too slow!

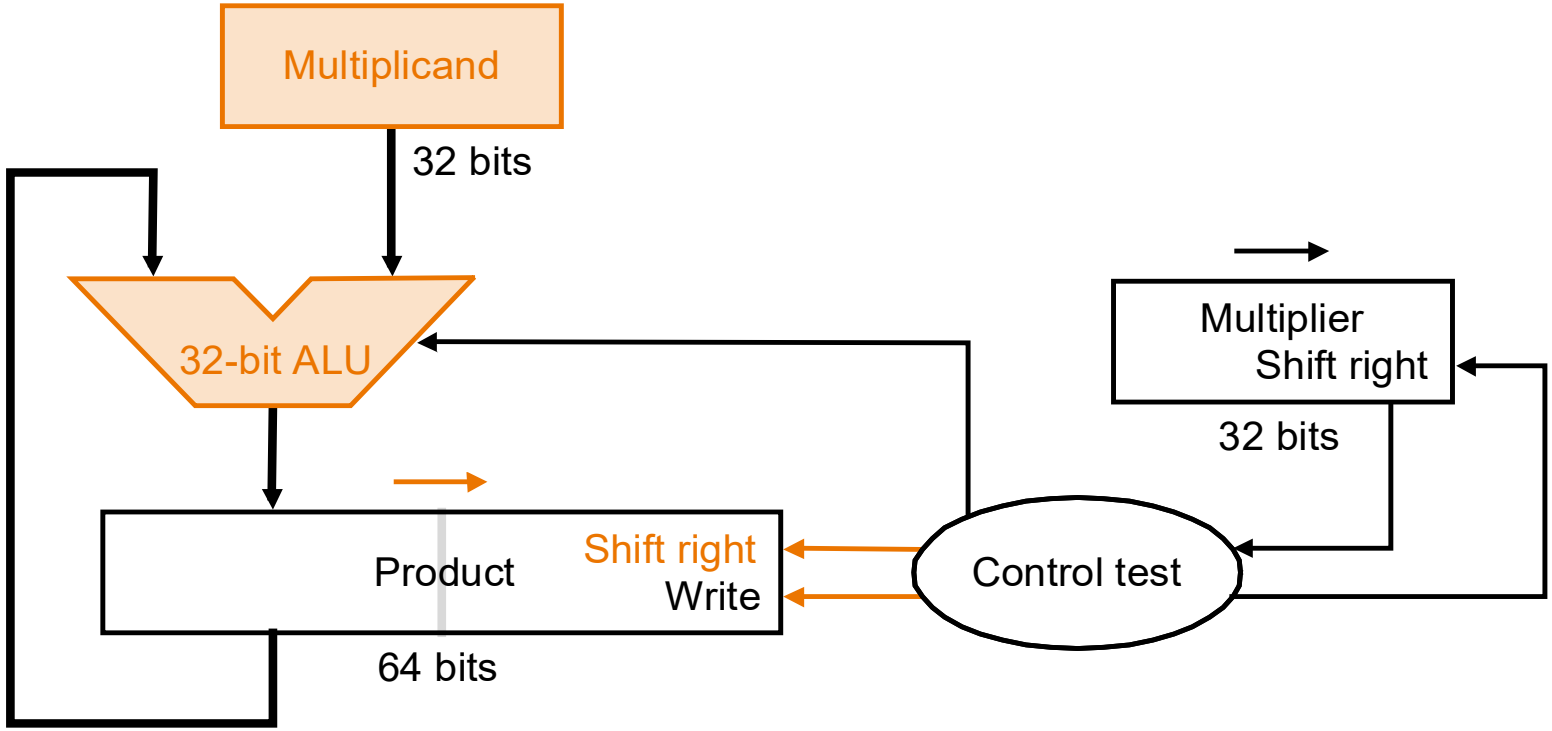


Multiplier V2

- Real addition is performed only with 32 bits
- Least significant bits of the product don't change
- **New idea:**
  - Don't shift the multiplicand
  - Instead, **shift the product**
  - Shift the multiplier
- ALU reduced to 32 bits!

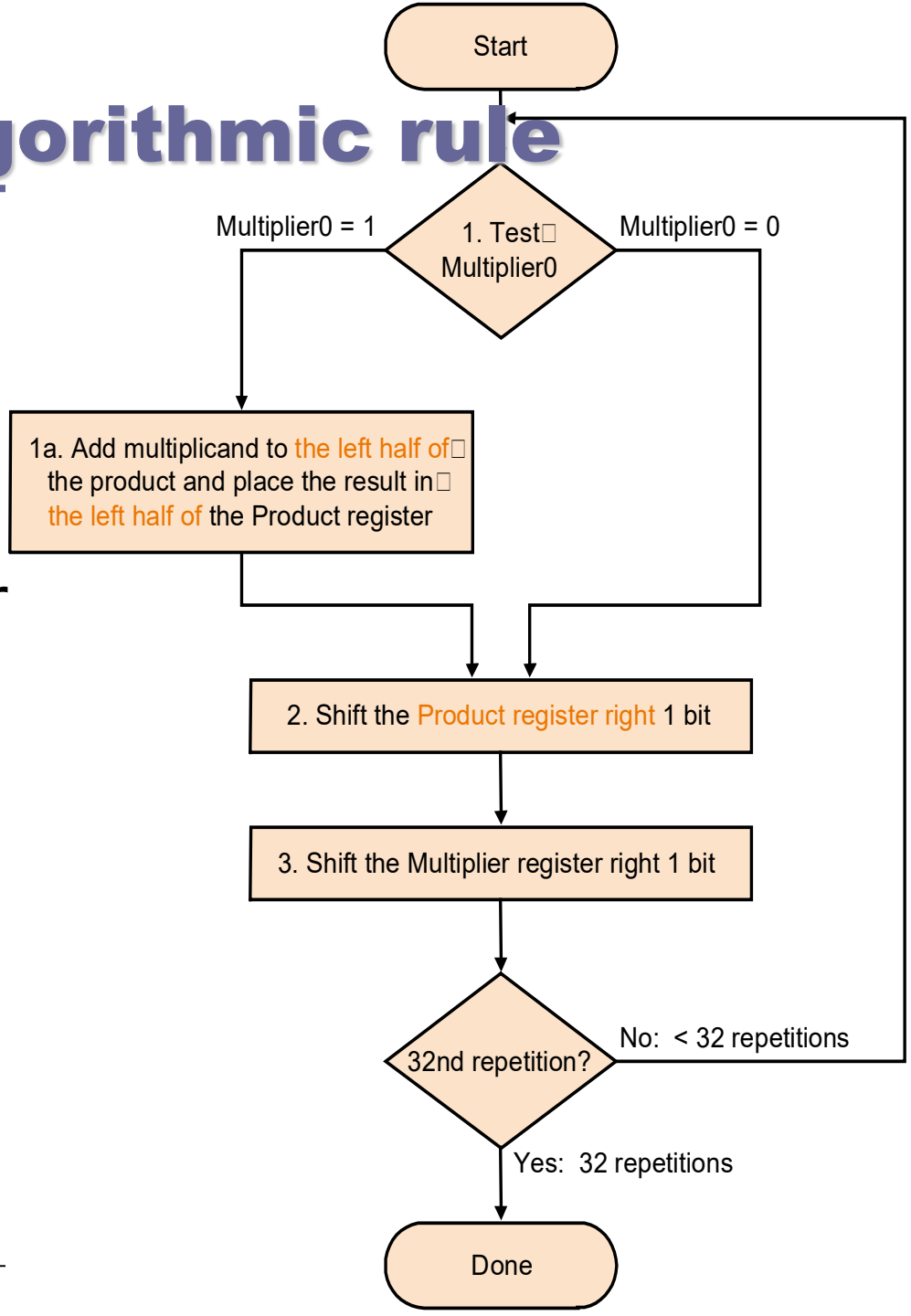
Multiplier V2-- Logic Diagram

- Diagram of the V2 multiplier
- Only **left half of product register is changed**



Multiplier V2----Algorithmic rule

- Addition performed only on left half of product register
- Shift of product register



Revised 4-bit example with V2

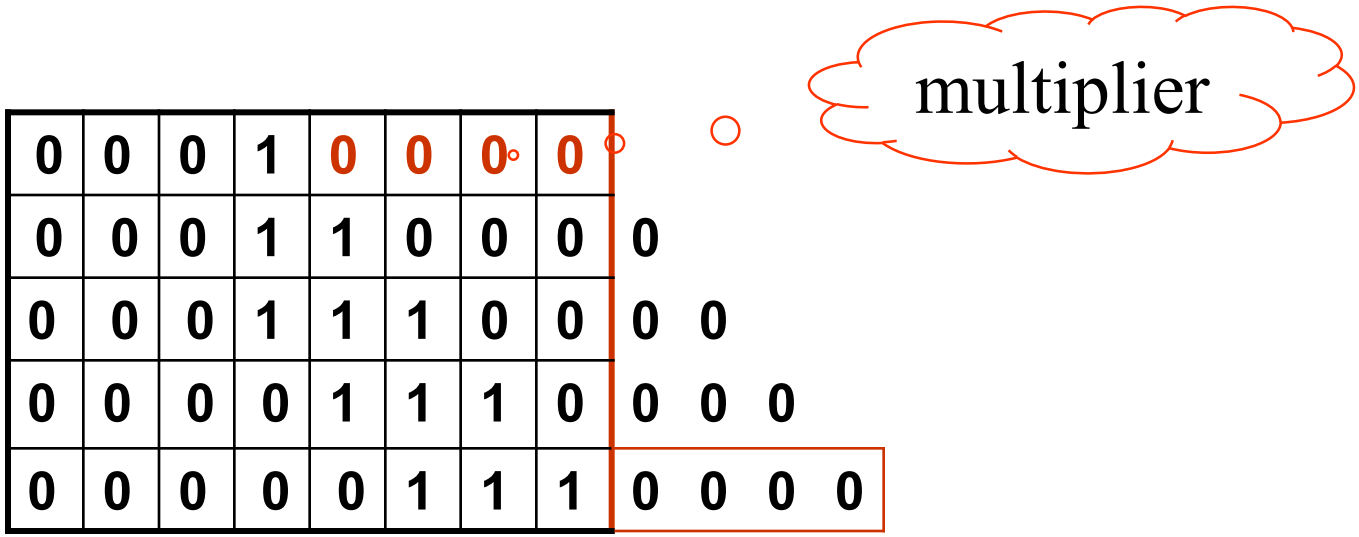
- Multiplicand x multiplier: 0001 x 0111

		Shift out	
Multiplicand:	**0001**		
Multiplier: ×	**0111**		
	00000000		#Initial value for the product
1	**00010000**		#After adding 0001, Multiplier=1
	00001000	**0**	#After shifting right the product one bit
	0001		
2	**00011000**		#After adding 0001, Multiplier=1
	00001100	**0**	#After shifting right the product one bit
	0001		#After adding 0001, Multiplier=1
3	**00011100**		
	00001110	**0**	#After shifting right the product one bit
	0000		
4	**00001110**		#After adding 0001, Multiplier=0
	00000111	**0**	#After shifting right the product one bit

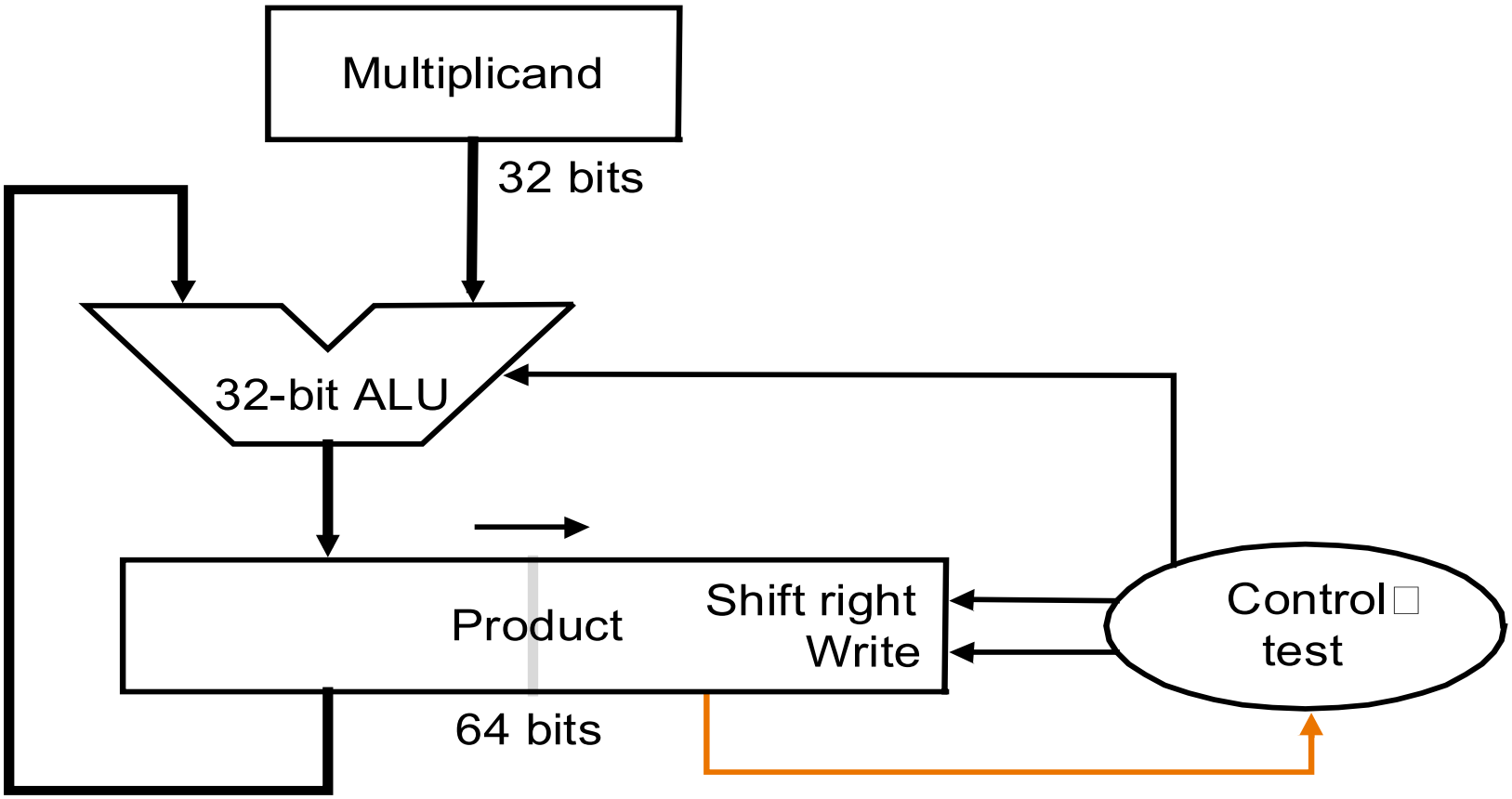
Multiplier V 3

- Further optimization
- At the initial state the product register contains only '0'
- The lower 32 bits are simply shifted out
- Idea:

  use these lower 32 bits for the multiplier

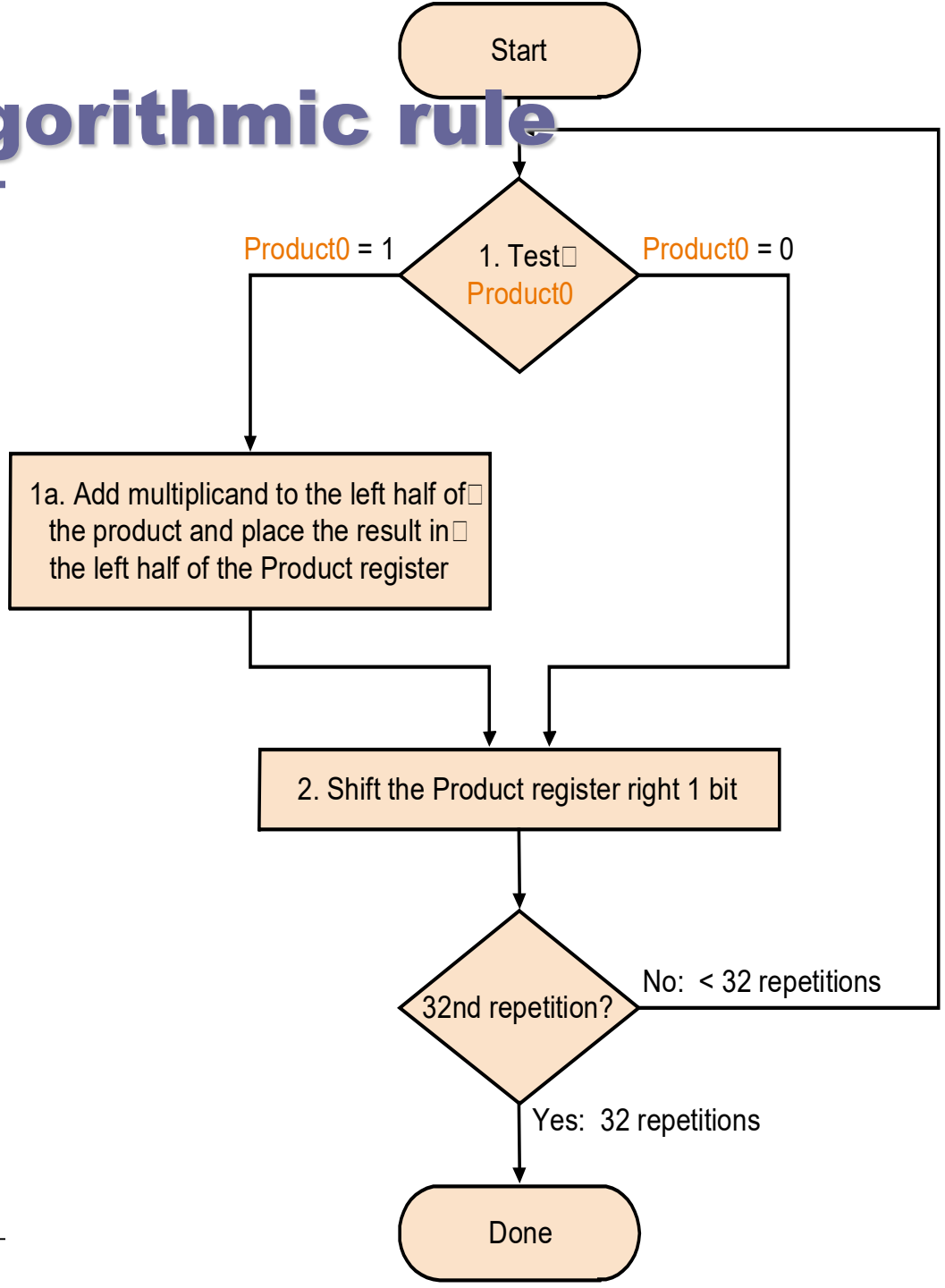


Multiplier V3 Logic Diagram



Multiplier V3--Algorithmic rule

- Set product register to '0'
- Load lower bits of product register with multiplier
- Test least significant bit of product register



Example with V3

- Multiplicand x multiplier: 0001 x 0111

		Shift out	
Multiplicand:	**0001**		
Multiplier: ×	**0111**		
	00000111		#Initial value for the product
1	**00010111**		#After adding 0001, Multiplier=1
	00001011	**1**	#After shifting right the product one bit
	0001		
2	**00011011**		#After adding 0001, Multiplier=1
	00001101	**1**	#After shifting right the product one bit
	0001		#After adding 0001, Multiplier=1
3	**00011101**		
	00001110	**1**	#After shifting right the product one bit
	0000		
4	**00001110**		#After adding 0001, Multiplier=0
	00000111	**0**	#After shifting right the product one bit

Multiply example using algorithm in Figure 3.5. （for V1）

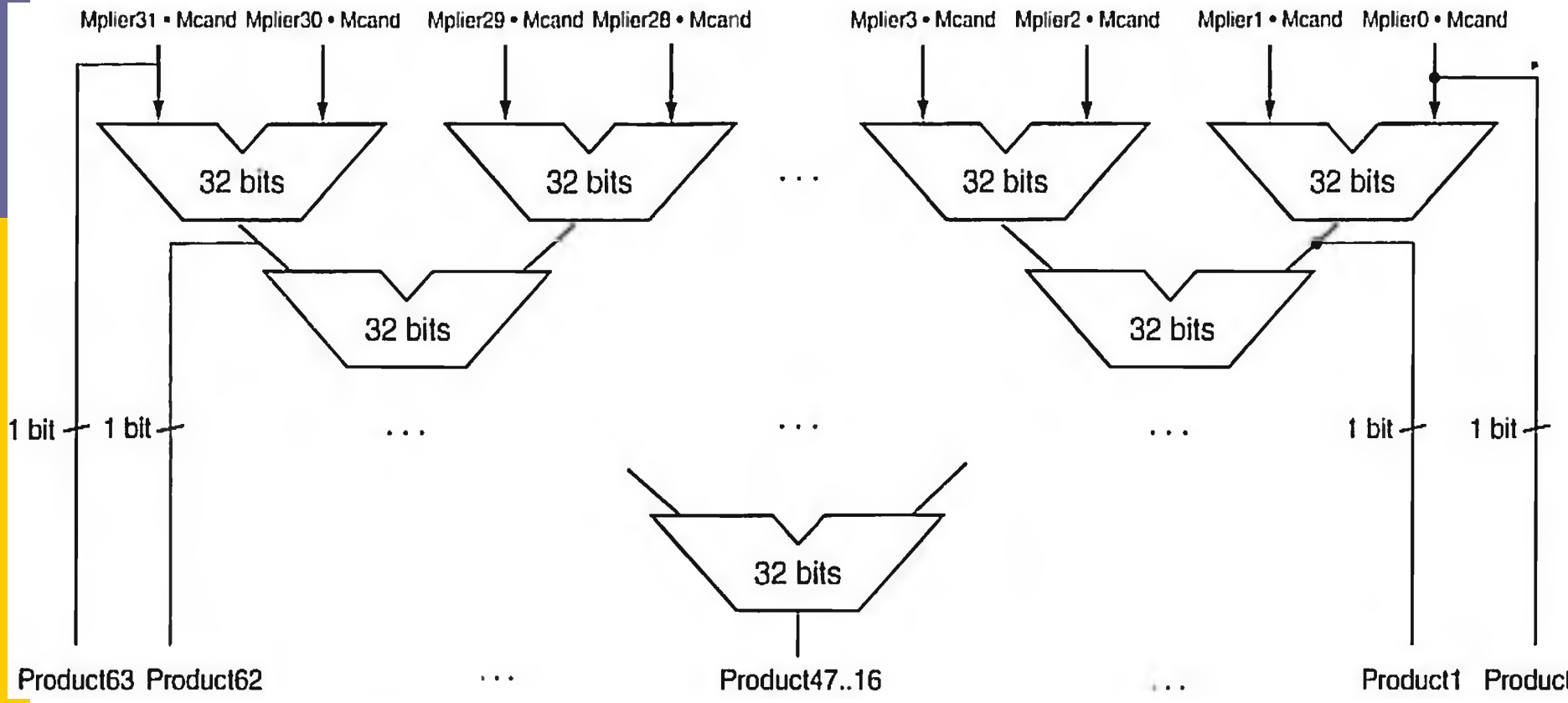
Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	0011	0000 0010	0000 0000
1	1a: 1 ⇒ Prod = Prod + Mcand	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	0001	0000 0100	0000 0010
2	1a: 1 ⇒ Prod = Prod + Mcand	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	0000	0000 1000	0000 0110
3	1: 0 ⇒ No operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	0000	0001 0000	0000 0110
4	1: 0 ⇒ No operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

Signed multiplication

- Basic approach:
  - Store the signs of the operands
  - Convert signed numbers to unsigned numbers (most significant bit (MSB) = 0)
  - Perform multiplication
  - If sign bits of operands are equal
    sign bit = 0, else
    sign bit = 1

Faster Multiplication

前面先测试乘数位是否为1，需要时间。不管乘数的某一位是否为1，都进行加法运算，只要位置正确。并可以并行进行。



Multiply in RISC-V

- MUL ,MULH(有符号)
- MULHU (无符号)

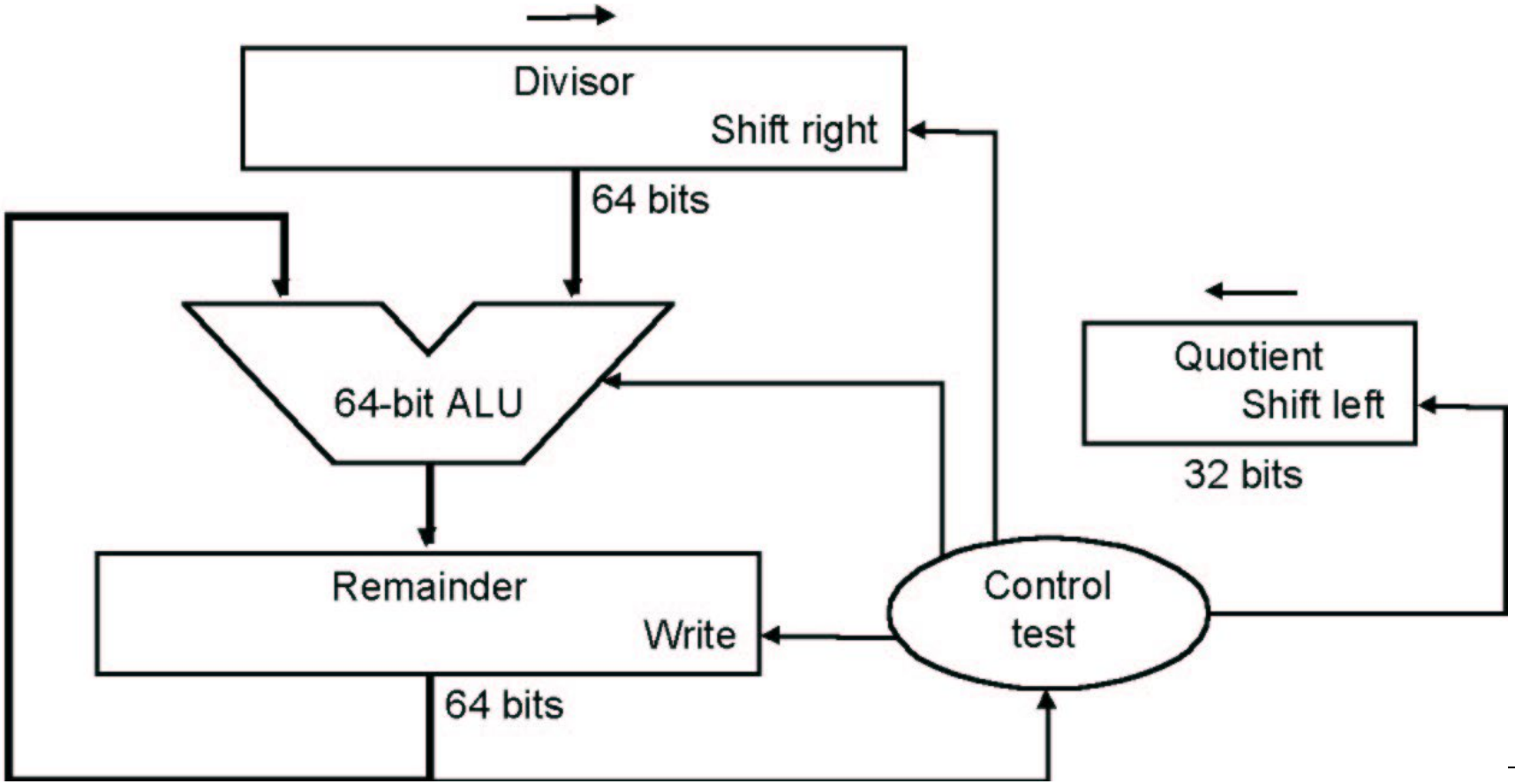
3.4 Division

- Dividend = quotient × divisor + remainder
  - Remainder < divisor
  - Iterative subtraction
- Result:
  - Greater than 0: then we get a 1
  - Smaller than 0: then we get a 0

$$
\begin{array}{rll}
 & 1001_{ten} & \text{Quotient} \\
\text{Divisor } 1000_{ten} \;\big\lceil & 1001010_{ten} & \text{Dividend} \\
 & -1000 & \\
 & \overline{10} & \\
 & 101 & \\
 & 1010 & \\
 & -1000 & \\
 & \overline{10_{ten}} & \text{Remainder}
\end{array}
$$

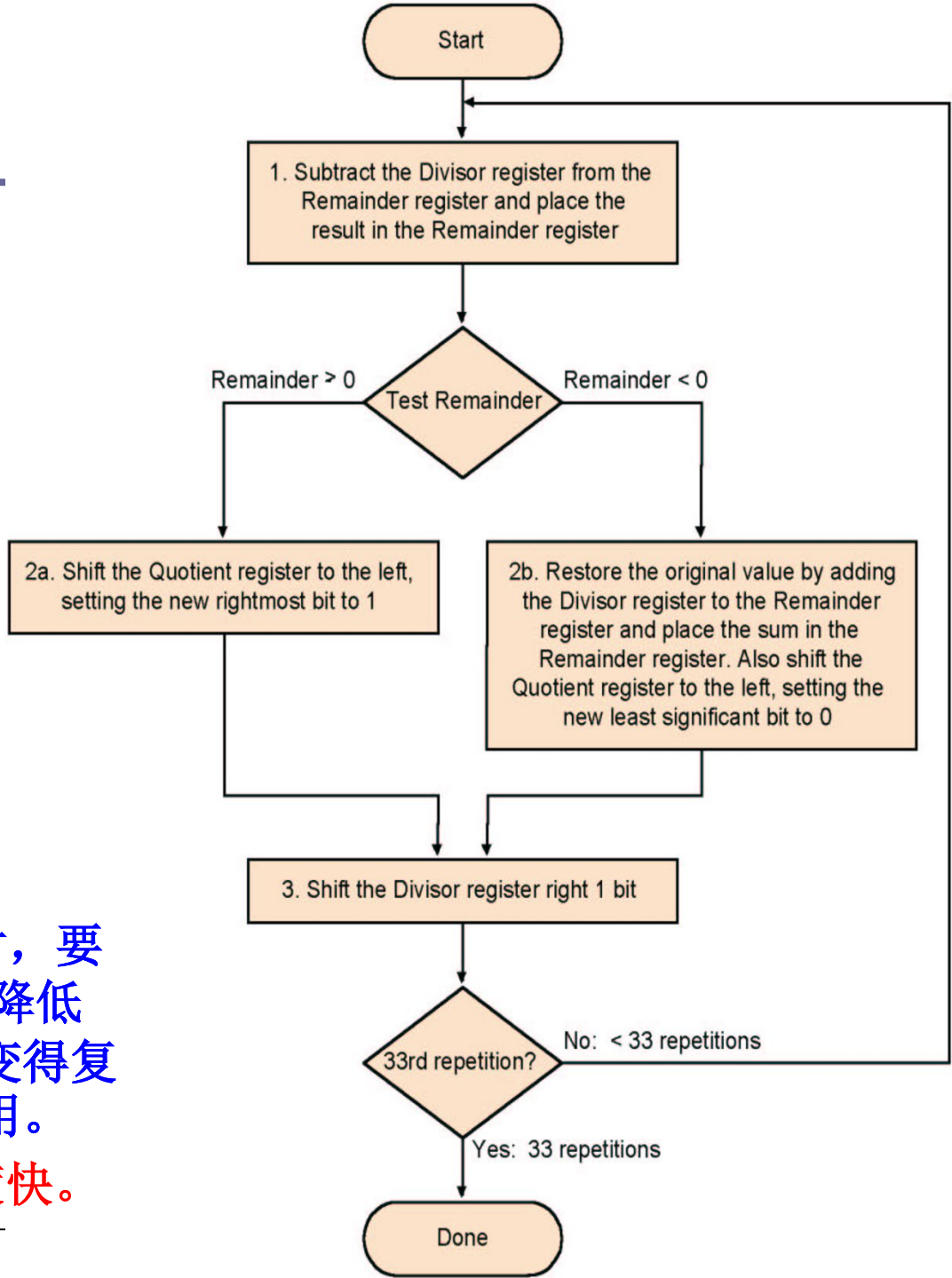
Division V1 --Logic Diagram（恢复余数方法）

- At first, the divisor is in the left half of the divisor register, the remainder register is initialized with the dividend.
- Shift right the divisor register each step



Algorithm V 1

- Each step:
  - Subtract divisor
  - Depending on Result
    - Leave or
    - Restore
  - Depending on Result
    - Write '1' or
    - Write '0`



缺点：当某一次-Y的差值为负时，要多一次+Y恢复余数的操作，降低了执行速度，又使控制线路变得复杂，因此在计算机中很少采用。

改进目标：电路简单，运算速度快。

Example 7/2 for Division V1

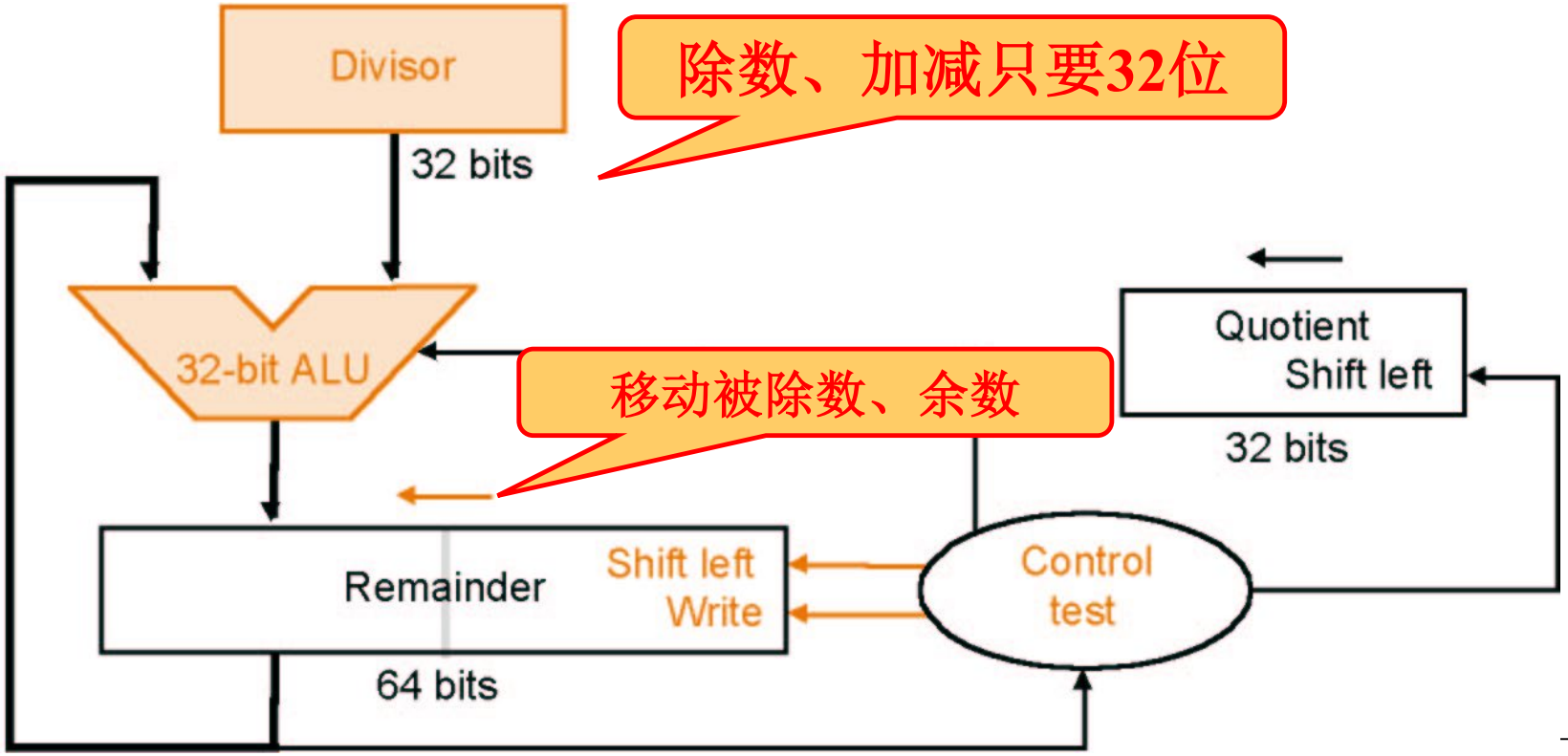
不够减，加上除数

判断是否够减

Iteration	Step	Quotient	Divisor	Remainder
0	Initial Values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	1110 0111
	2b: Rem<0 => +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	1111 0111
	2b: Rem < 0 => +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	1111 1111
	2b: Rem < 0 => +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	0000 0011
	2a: Rem 0 => sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	0000 0001
	2a: Rem 0 => sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: shift Div right	0011	0000 0001	0000 0001

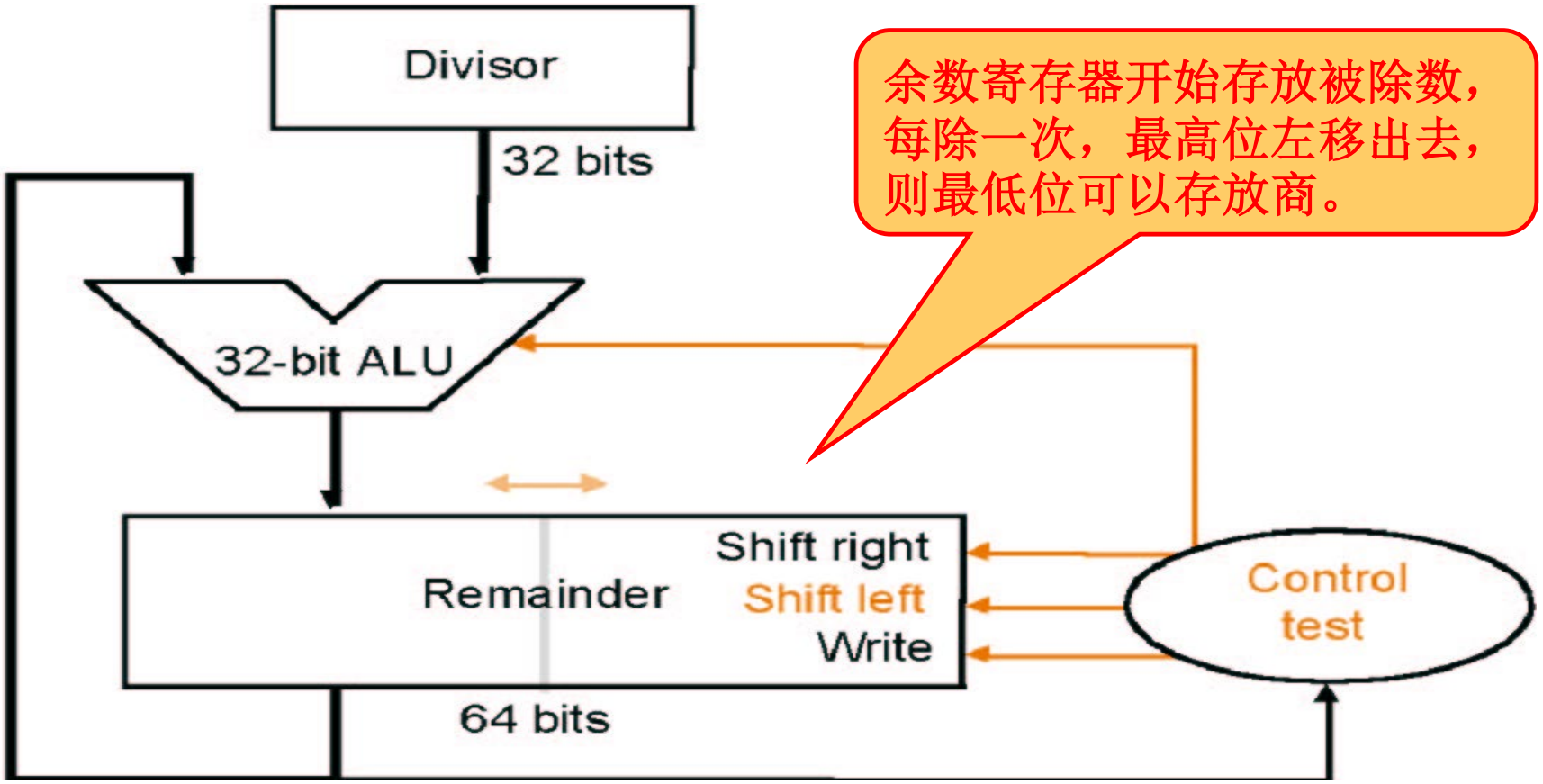
Division V 2

- Reduction of Divisor and ALU width by half
- Shifting of the remainder
- Saving 1 iteration



Division V 3

- Remainder register keeps quotient
  No quotient register required



Signed division

- Keep the signs in mind for Dividend and Remainder
  - (+ 7) ÷(+ 2) = + 3 Remainder = +1
  - 7 = 3 × 2 + (+1) = 6 + 1
  - (- 7) ÷(+ 2) = - 3 Remainder = -1
  - -7 = -3 × 2 + (-1) = - 6 - 1
  - (+ 7) ÷(- 2) = - 3 Remainder = +1
  - (- 7) ÷(- 2) = + 3 Remainder = -1

  - （+7）÷(- 2) = - 4 Remainder = -1

在符号数除法时，为使商唯一确定，规定余数符号必须与被除数一致。当被除数与除数符号不同时，商的符号取被除数符号的反

Divide in RISC-V

- <op> Rn, Rm
  - <op> is DIV (signed) or DIVU (unsigned)

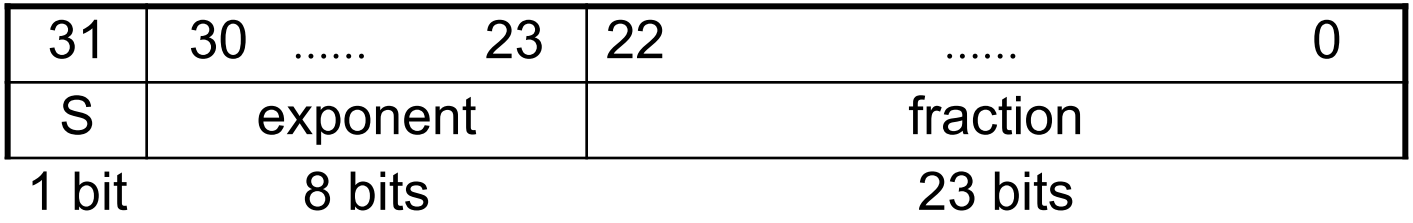
3.5 Floating point numbers

- Reasoning
  - Larger number range than integer rage
  - Fractions
  - Numbers like e (2.71828) and π (3.14159265....)
- Representation
  - Sign
  - Significant (fraction尾数)
  - Exponent
  - More bits for significant: more accuracy
  - More bits for exponent: increases the range

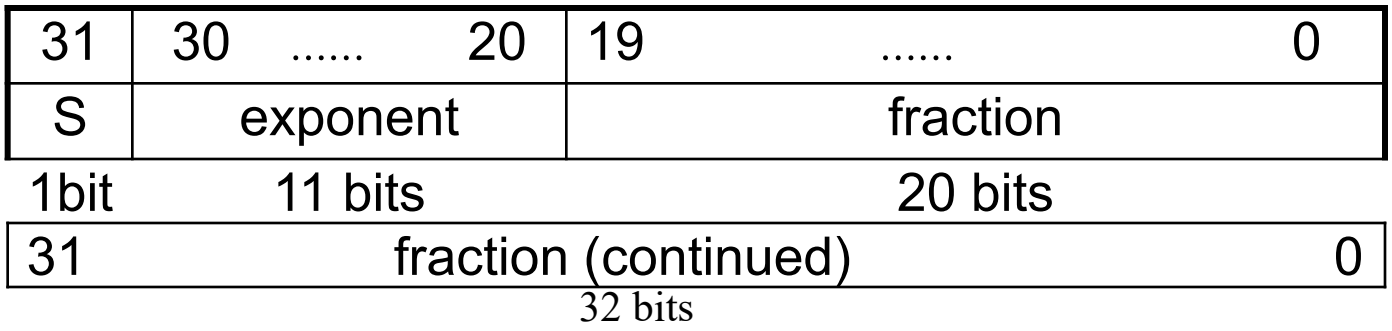
Floating point numbers

- Form
  - Arbitrary $363.4 \cdot 10^{34}$
  - Normalized $3.634 \cdot 10^{36}$ (标准化)
- Binary notation
  - Normalized $1.xxxxxx \cdot 2^{yyyyy}$ （标准化）
- Standardized format IEEE 754, C99开始支持该标准
  - Single precision 8 bit exp, 23 bit significant（31位）
  - Double precision 11 bit exp, 52 bit significant（63位）
  - 最高位是符号位
  - 8位指数包含符号位。

- Single precision



- Double precision



IEEE 754 standard

- Leading '1' bit of significand is implicit
  ->saves one bit（小数点前的1可以不写）
- Exponent is biased: 移码

指数有正有负，看起来不习惯，用移码，即都加上一个数,使:

00...000 smallest exponent

11...111 biggest exponent

这个数为（偏移值）:

- Bias 127 for single precision
- Bias 1023 for double precision

- Summary:

$$(-1)^{sign} \cdot (1 + significant) \cdot 2^{exponent - bias}$$

IEEE 754 standard

- Rounding: four rounding modes **(舍入、舍去模式)**
  - Round to next even number (default)（最接近的偶数）
  - Round to 0(truncate, 总是舍去）
  - Round to +∞ （总是向上舍入）
  - Round to -∞ （总是向下舍入）
- Special numbers: NaN, +∞ , -∞
- denormal numbers for results smaller $1.0 \cdot 2^{Emin}$
- Mechanisms for handling exceptions

Example

- Show the binary representation of -0.75 in IEEE single precision format
- Decimal representation: $-0.75 = -3/4 = -3/2^2$
- Binary representation: $-0.11 = -1.1 \cdot 2^{-1}$
- Floating point
  - $(-1)^{sign} \cdot (1 + fraction) \cdot 2^{exponent - bias}$
  - $(-1)^{sign} = -1$, so Sign = 1
  - 1+ fraction = 1.1, so Significand=.1
  - exponent -127 =-1, so Exponent=(-1 + 127) = 126

Single precision

31	30 23	22 0
1	0111 1110	100 0000 0000 0000 0000 0000
1 bit	8 bits	23 bits

+1023

Double precision

31	30 20	19 0
1	011 1111 1110	1000 0000 0000 0000 0000
1bit	11 bits	20 bits

0000	0000	0000	0000	0000	0000	0000	0000

Limitations

- Overflow:

  The number is too big to be represented
- Underflow:

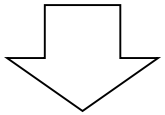
  The number is too small to be represented

- 00... 00_{two} represents 0;
- instead of interrupting on a divide by 0, software can set the result to a bit pattern representing +∞ or -∞;
- *NaN*: Not a Number.
- ±∞ - represented by f=0, E=255, S=0,1 - must obey all mathematical conventions: F+∞=∞, F/∞=0
- Denormalized numbers - represented by E=0 -values smaller than smallest normalized number - lowering probability of exponent underflow

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	± denormalized number
1–254	Anything	1–2046	Anything	± floating-point number
255	0	2047	0	± infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

Converting Binary to Decimal Floating Point

31	30 23	22 0
1	1000 0001	010 0000 0000 0000 0000 0000
1 bit	8 bits	23 bits



$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})} = (-1)^1 \times (1 + 0.25) \times 2^{(129-127)}$$
$$= -1 \times 1.25 \times 2^2$$
$$= -1.25 \times 4$$
$$= -5.0$$

Floating point addition

- Alignment

（因为指数有大小，加前需小数点对齐）

- The proper digits have to be added
- Addition of significants
- Normalisation of the result
- Rounding
- Example in decimal

system precision 4 digits

What is $9.999 \cdot 10^{1} + 1.610 \cdot 10^{-1}$?

Example for Decimal

- Aligning the two numbers

  $9.999 \cdot 10^1$

  $0.01610 \cdot 10^1 \rightarrow 0.016 \cdot 10^1$ Truncation

  (舍去多余位)

- Addition

$$\begin{array}{rl} 9.999 & \cdot 10^1 \\ \underline{+\ 0.016} & \underline{\cdot 10^1} \\ 10.015 & \cdot 10^1 \end{array}$$

- Normalization

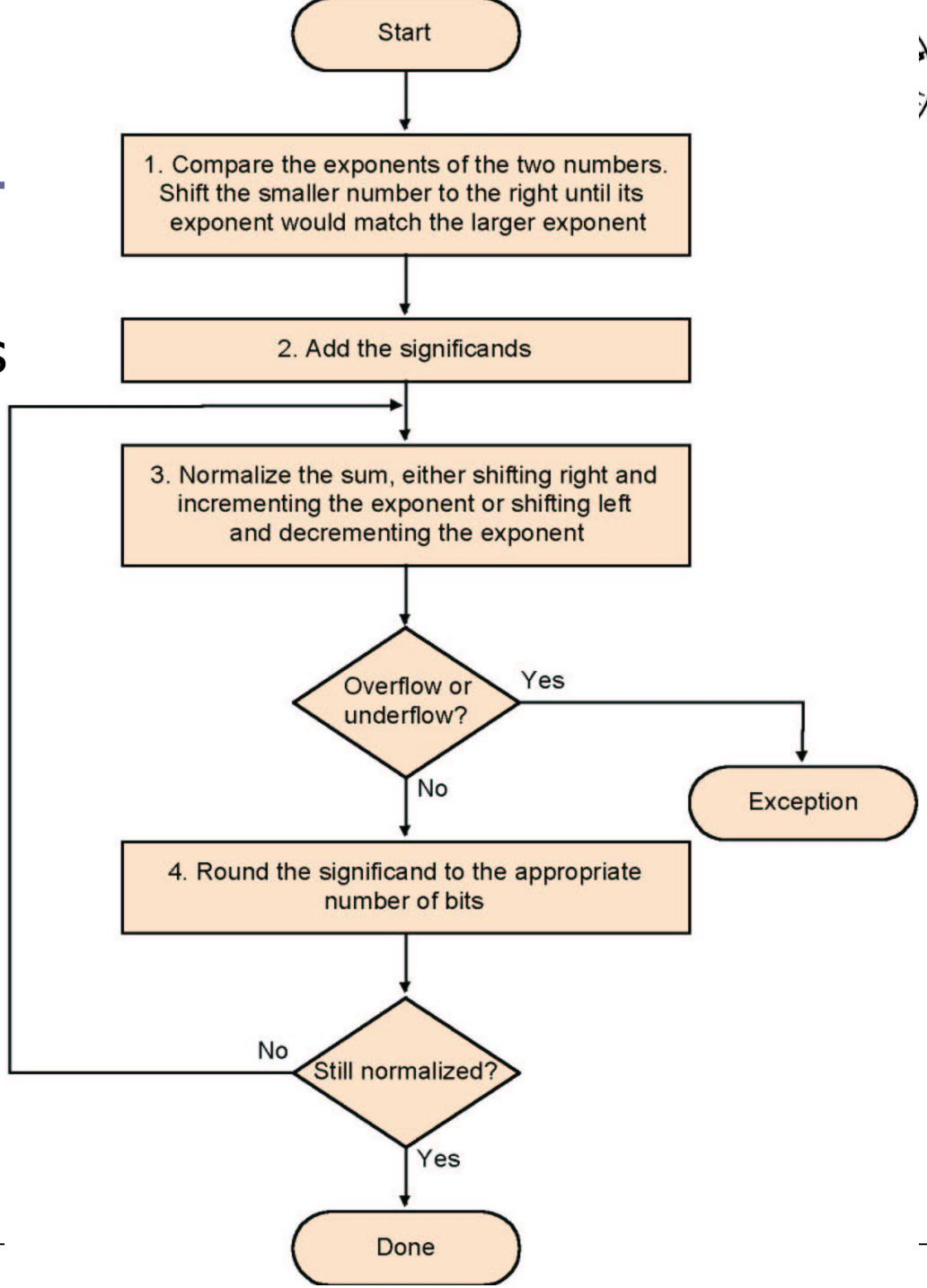
  $1.0015 \cdot 10^2$

- Rounding

  $1.002 \cdot 10^2$

Algorithm

- Normalise Significands
- Add Significands
- Normalise the sum
- Over/underflow
- Rounding
- Normalisation



Example y=0.5+(-0.4375) in binary

- $0.5_{10} = 1.000_2 \times 2^{-1}$
- $-0.4375_2 = -1.110_2 \times 2^{-2}$
- Step1:The fraction with lesser exponent is shifted right until matches

  $-1.110_2 \times 2^{-2} \rightarrow -0.111_2 \times 2^{-1}$
- Step2: Add the significands

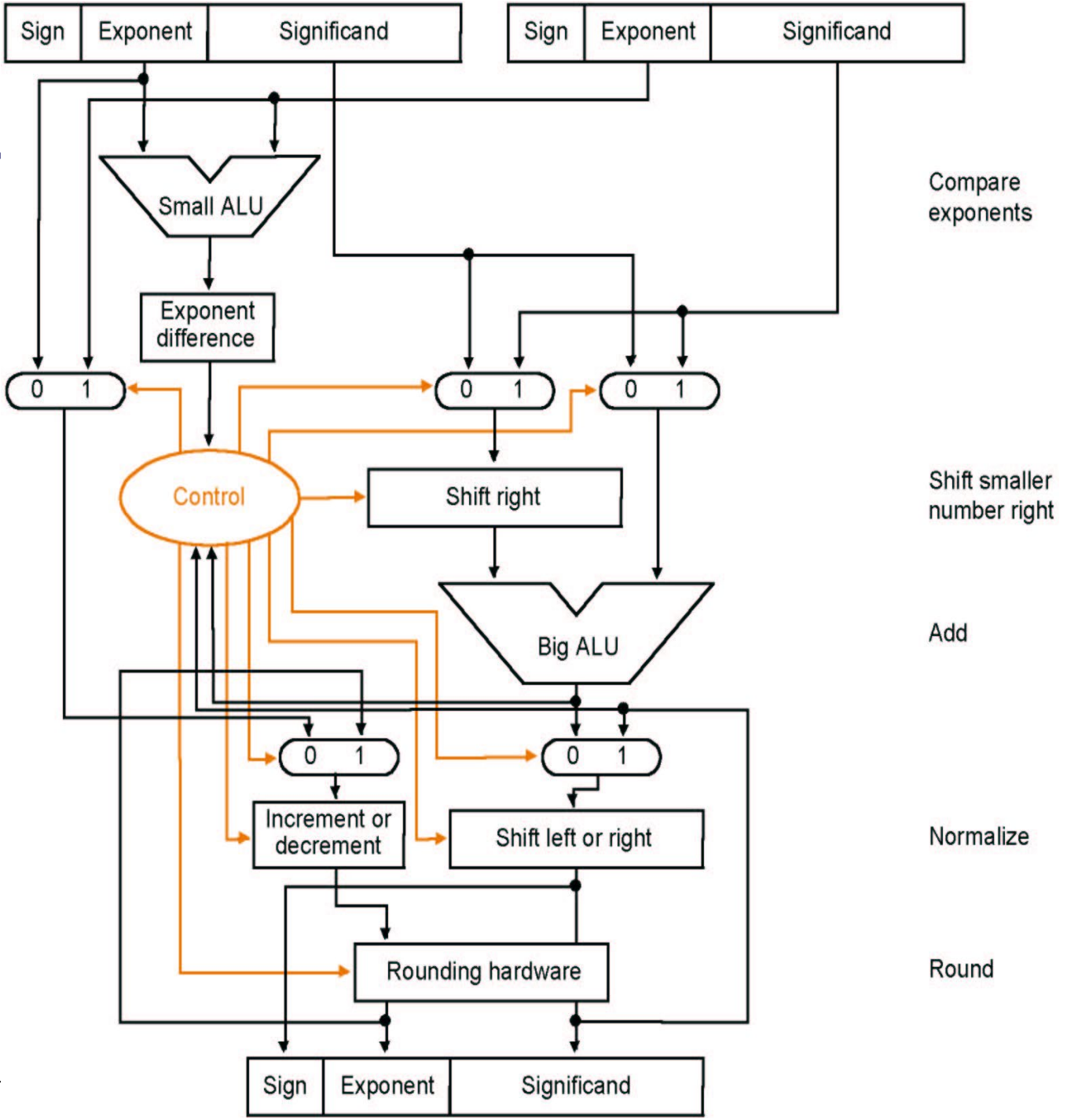
$$\begin{array}{r} \mathbf{1.000_2 \times 2^{-1}} \\ \mathbf{+) \; -0.111_2 \times 2^{-1}} \\ \hline \mathbf{0.001_2 \times 2^{-1}} \end{array}$$

- Step3: Normalize the sum and checking for overflow or underflow

  $\mathbf{0.001_2 \times 2^{-1} \rightarrow 0.010_2 \times 2^{-2} \rightarrow 0.100_2 \times 2^{-3} \rightarrow 1.000_2 \times 2^{-4}}$
- Step4: Round the sum

  $\mathbf{1.000_2 \times 2^{-4} = 0.0625_{10}}$

Algorithm


Sign
Exponent
Significand
Sign
Exponent
Significand
Small ALU
Compare exponents
Exponent difference
0 1
0 1
0 1
Control
Shift right
Shift smaller number right
Big ALU
Add
0 1
0 1
Increment or decrement
Shift left or right
Normalize
Rounding hardware
Round
Sign
Exponent
Significand

Multiplication

- Composition of number from different parts

  → separate handling

  $$(s1 \cdot 2^{e1}) \cdot (s2 \cdot 2^{e2}) = (s1 \cdot s2) \cdot 2^{e1+e2}$$

- Example

  1 10000010 000 0000 0000 0000 0000 0000 = -1×2^3

  0 10000011 000 0000 0000 0000 0000 0000 = 1×2^4

- Both significands are 1 → product = 1 →Sign=1
- Add the exponents, bias = 127

  10000010

  +10000011

  110000101

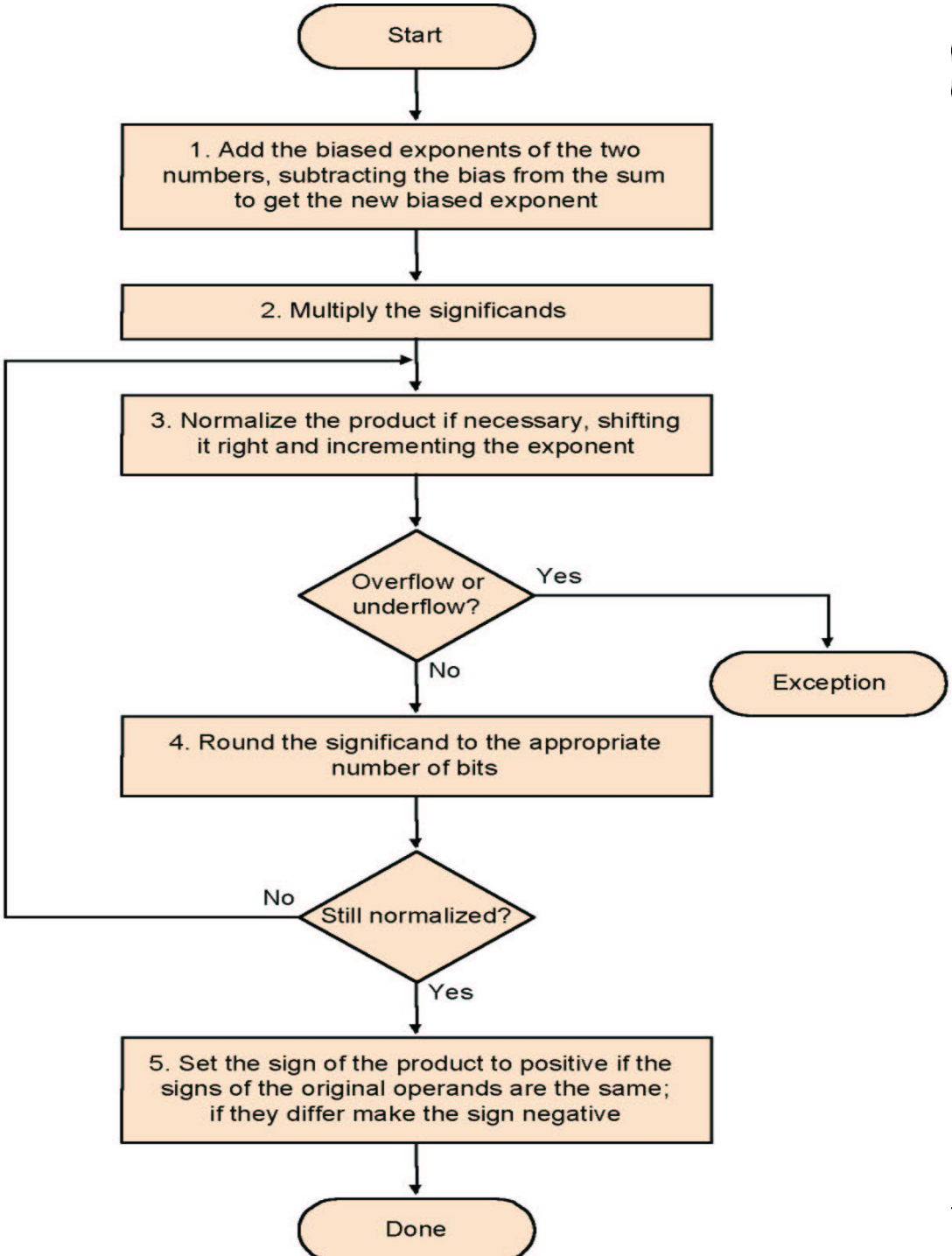
  加了2个127，要减去1个。

  Correction: 110000101-01111111=10000110=134=127+3+4

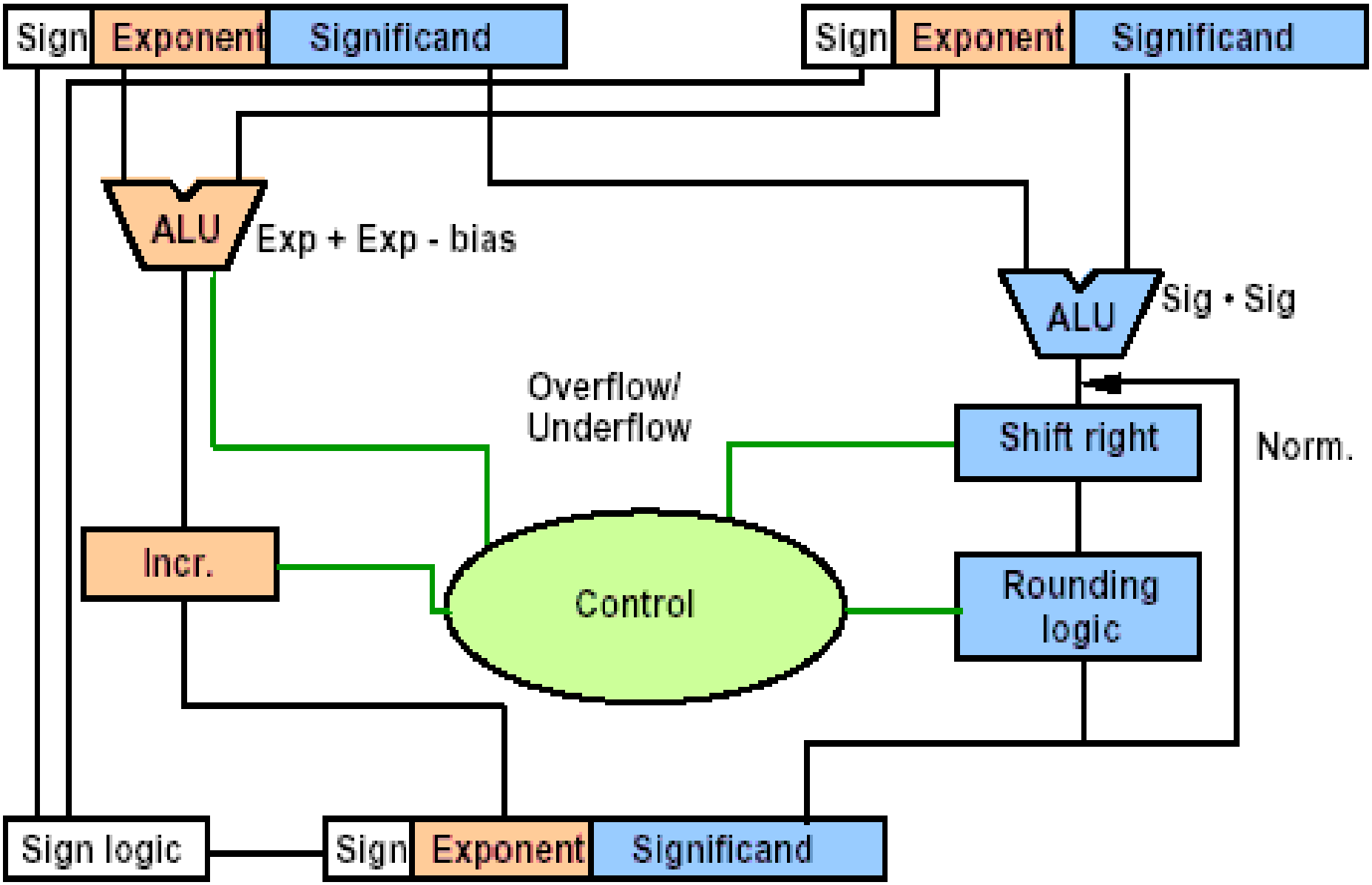
- The result: 1 10000110 000 0000 0000 0000 0000 0000 = -1×2^7

Multiplication

- Add exponents
- Multiply the significands
- Normalize
- Over- underflow
- Rounding
- Sign



Data Flow


Sign
Exponent
Significand
Sign
Exponent
Significand
ALU
Exp + Exp - bias
ALU
Sig • Sig
Overflow/
Underflow
Shift right
Norm.
Incr.
Control
Rounding
logic
Sign logic
Sign
Exponent
Significand

multiplying the numbers 0.5_{ten} and $-0.4375_{ten} \rightarrow 1.000_{two} \times 2^{-1}$ by $-1.110_{two} \times 2^{-2}$

- Step1:Adding the exponents without bias or using the biased
  - -1 + (-2)=-3
  - (-1 + 127) + (-2 + 127) - 127 = (-1 - 2) + (127 + 127-127) =-3 + 127= 124
- Step 2. Multiplying the significands
  - $1.110000_{two} \times 2^{-3}$
- Step 3. normalize
  - 127 ≥ -3 ≥ -126, no overflow or underflow.
- Step 4. Rounding
  - $1.110_{two} \times 2^{-3}$
- Step 5. sign
  - $-1.110_{two} \times 2^{-3} = -0.21875_{ten}$

$$
\begin{array}{rl}
 & 1.000_{two} \\
\times & 1.110_{two} \\
\hline
 & 0000 \\
 & 1000 \\
 & 1000 \\
 & 1000 \\
\hline
 & 1110000_{two}
\end{array}
$$

Division-- Brief

- Subtraction of exponents
- Division of the significants
- Normalisation
- Runding
- Sign

Floating point instruction in RISC-V

- instruction
  - Add（single，double）：FADD.S，FADD.D
  - Sub（single，double）：FSUB.S，FSUB.D
  - Multiplication（single，double）：FMUL.S，FMUL.D
  - division（single，double）：FDIV.S，FDIV.D
  - Square root （single，double）：FSQRT.S，FSQRT.D
  - comparision（single，double）：FX.S，FX.D （X可以是eq, neq, lt, le,）
- FLW,FLD（single，double）：
  - Float load
- FSW,FSD （single，double）：
  - Float store

Compiling a Floating-Point C Program into RISC-V Assembly Code

华氏温度转换为摄氏温度

- C code

```
float f2c (float fahr)
{
    return ((5.0/9.0) * (fahr- 32.0));
}
```

- RISC-V ASM code （P.208)

F2c:

1、先把5.0和9.0放到浮点数寄存器；

2、得到5.0/9.0数；

3、load 32,并从fahr中减去32；

4、二个数相乘并存储，作为返回值。

Compiling Floating-Point C Procedure with Two-Dimensional Matrices into RISC-V

- X = X + Y * Z
  - XYZ: 32 elements
- C code

```
void mm (double x[][], double y[][], double z[][])
{
        int i, j, k.
        for (i = 0: i < 32; i = i + 1)
                for (j = 0; j < 32; j = j + 1)
                        for (k = 0; k < 32: k = k + 1)
                                x[i][j] = x[i][j] + y[i][k] * z[k][j];
}
```

- RISC-V ASM code

  P.210-211

Accurate Arithmetic

- IEEE 754 always keeps two extra bits on the right during intermediate additions, called guard and round
- Rounding with Guard Digits
  - Add $2.56_{ten} \times 10^0$ to $2.34_{ten} \times 10^2$

$$\begin{array}{r} 2.3400_{ten} \\ +\ 0.0256_{ten} \\ \hline 2.3656_{ten} \end{array}$$

  - The guard digit holds 5 and the round digit holds 6.
  - Sum=$2.37_{ten} \times 10^2$.
- Rounding without Guard Digits

$$\begin{array}{r} 2.34_{ten} \\ +\ 0.02_{ten} \\ \hline 2.36_{ten} \end{array}$$

  - Sum=$2.36_{ten} \times 10^2$

IEEE 754 round modes

- Directed roundings
  - Round toward 0 – directed rounding towards zero (also called truncation,截取)
  - Round toward +∞ – directed rounding towards positive infinity
  - Round toward −∞ – directed rounding towards negative infinity.
- Roundings to nearest
  - Round to nearest, ties to even – rounds to the nearest value; if the number falls midway it is rounded to the nearest value with an even (zero) least significant bit, which occurs 50% of the time; this is the default algorithm for binary floating-point and the recommended default for decimal
  - Round to nearest, ties away from zero – rounds to the nearest value; if the number falls midway it is rounded to the nearest value above (for positive numbers) or below (for negative numbers)

y	round down (towards −∞)	round up (towards +∞)	round towards zero	round away from zero	round to nearest
+23.67	+23	+24	+23	+24	+24
+23.50	+23	+24	+23	+24	+23 or +24
+23.35	+23	+24	+23	+24	+23
+23.00	+23	+23	+23	+23	+23
0	0	0	0	0	0
−23.00	−23	−23	−23	−23	−23
−23.35	−24	−23	−23	−24	−23
−23.50	−24	−23	−23	−24	−23 or −24
−23.67	−24	−23	−23	−24	−24